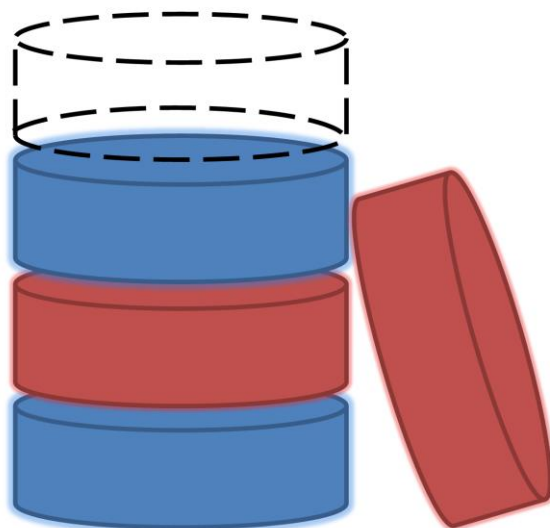


Universitatea Babeș-Bolyai
Facultatea de Matematică și Informatică

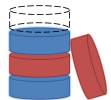


Baze de date

Suport de curs

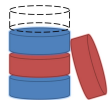
Lect. dr. Dan Mircea Suci

Cluj-Napoca, 2018-2022



Cuprins

1. Baze de date. Definiții și modele de date.....	3
2. Modelul relațional.....	6
3. Interogări în SGBD-urile relaționale	9
4. Dependente funcționale	11
5. Descompunerea relațiilor	16
6. Forme normale.....	19
7. Algebra relațională.....	25
8. Indexarea bazelor de date relaționale.....	29
9. Organizarea arborescentă a indecșilor	33
10. Organizarea directă a indecșilor.....	44
11. Structura fizică a bazelor de date	52
Bibliografie	60



1. Baze de date. Definiții și modele de date.

O **bază de date** este o colecție de date de dimensiuni mari, stocată pe o perioadă îndelungată de timp în scopul analizei ulterioare. De asemenea, o bază de date modelează aspecte ale lumii reale sau conceptuale folosind un *model de date*.

Cu alte cuvinte, bazele de date sunt utile în stocarea și gestionarea datelor. Bazele de date sunt disponibile pe orice tip de dispozitiv de calcul/sistem de operare și accesarea datelor dintr-o bază de date reprezintă una dintre cele mai frecvente utilizări a unui calculator.

Utilizăm zi de zi o mare diversitate de baze de date fără a conștientiza acest lucru. Ori de câte ori căutăm un număr de telefon, realizăm o tranzacție într-un cont bancar, platim un produs sau serviciu cu cardul sau achiziționăm un produs de pe internet, folosim o bază de date.

Mai mult decât atât, bazele de date nu reprezintă un concept legat de un sistem de calcul. Ca exemple de baze de date “*necomputerizate*” amintim: cartea de telefon, dicționarele, almanahurile etc. (deși în accepțiunea curentă acestea sunt exemple de rapoarte în format imprimat generate dintr-o bază de date).

Model de date

Un **model de date** este o colecție de concepte utilizate în descrierea datelor. Aceste concepte sunt utilizate în definirea structurii, semanticii, constrângerilor de consistență a datelor sau pentru definirea relațiilor cu alte date.

O **structură/schemă** este descrierea componentelor unei colecții de elemente/date utilizând un model particular. Structura este în general definită la începutul dezvoltării unei aplicații software și este rareori modificată ulterior.

O **instanță** a unei baze de date este formată din datele/valorile efective stocate într-o structură definită. Datele se pot schimba frecvent și pot fi comparate cu “*variabilele cu tip*” din limbajele de programare de nivel înalt.

Exemple:

- Modelul de date este un *graf*. De exemplu, nodurile pot reprezenta orașe iar arcurile dintre noduri rute aeriene
- Modelul de date este un *document XML*. De exemplu, documentul poate conține lista cărților cu cod ISBN ca identificator, titlu și numele autorului/autorilor ca sub-elemente
- Modelul de date este o *mulțime de înregistrări*. De exemplu, înregistrările pot conține cod de student, nume, adresă, listă cursuri, fotografie.

Primul model de date important a fost *Modelul Ierarhic*. El a fost definit spre finalul anilor ‘60 și a reprezentat o extindere a unui sistem de stocare/procesare a fișierelor. O structură ce utilizează acest model organizează datele într-o structură arborescentă.

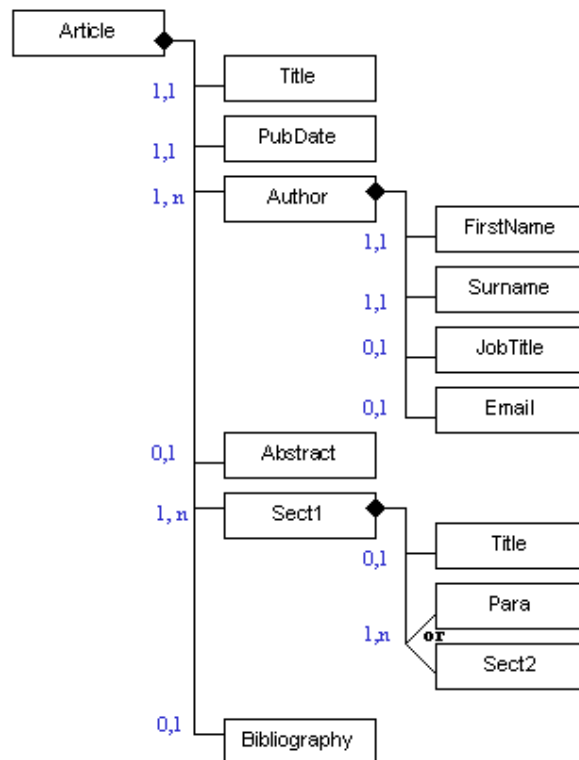
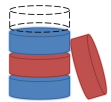
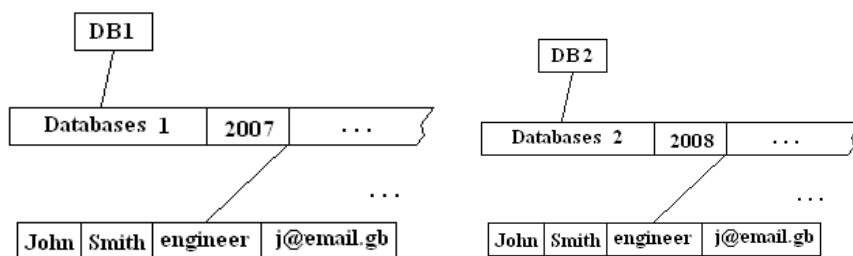
Figura 1.1 Structura unei entități *Article* folosind *Modelul Ierarhic*

Figura 1.2 Două instanțe particulare ale structurii din Fig. 1.1

Un alt model de date important este *Modelul Rețea* care reprezintă o extensie a *Modelului Ierarhic*, organizând datele într-o structură de tip graf. Figura 1.3 prezintă structura unei baze de date folosind *Modelul Rețea*.

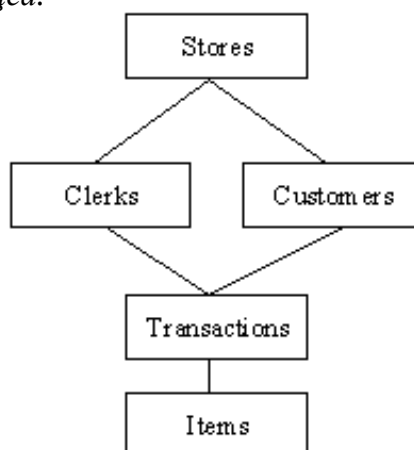


Figura 1.3. Bază de date proiectată folosind modelul rețea



La începuturile anilor '70 Ted Codd învețează *Modelul Relational* și conceptul de abstractizare a datelor. Modelul relațional este cel mai popular model de date în zilele noastre și va reprezenta principalul subiect al cursului curent.

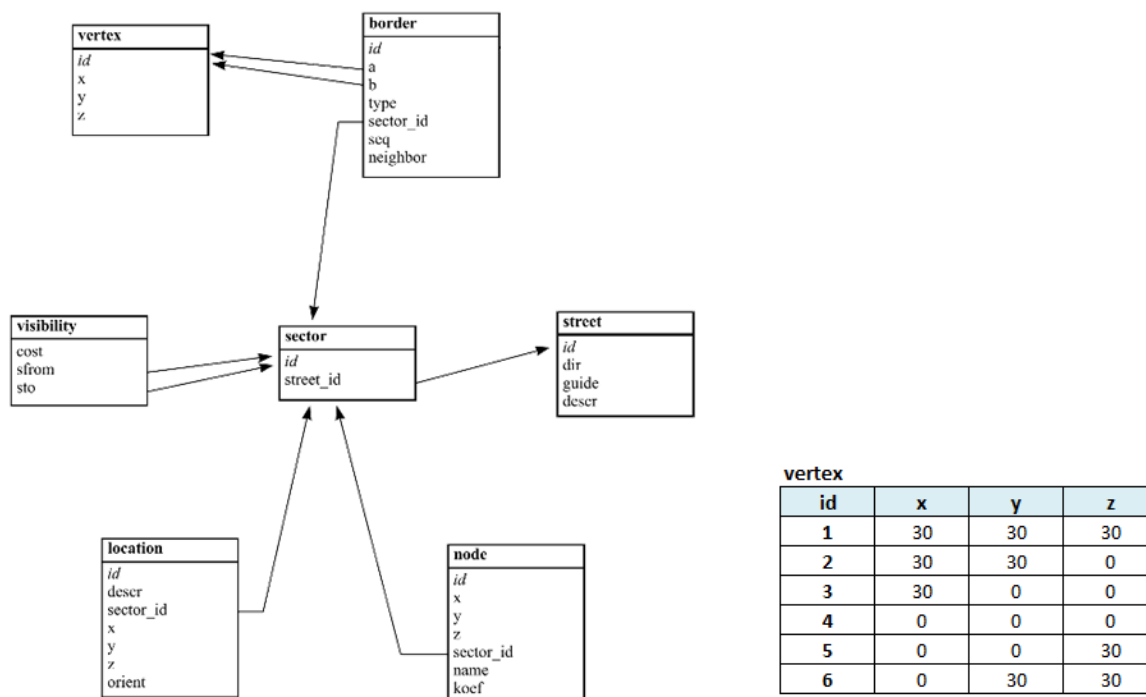


Figura 1.4 Exemplu de structură de baze de date și o instanță a unui tabel folosind modelul relațional

Modelul Orientat-Obiect – introduce concepte (ca clasă, atribut, metodă) și relații între acestea (asocieri, agregări, moștenire). Modelul orientat-obiect este foarte popular ca filozofie de analiză, proiectare și dezvoltare de soft. În baze de date, din motive de eficiență, el prezintă doar un interes “științific”.

Structură vs. Date

Structura bazelor de date

- Se modifică rar;
- Denumită și *metadata* (= date reprezentând date);

Starea bazei de date

- Se modifică frecvent;
- Sistemul de gestiune a bazelor de date asigură că starea fiecărei baze de date este una *validă*.

Instanța bazei de date se referă în general la combinația dintre structură și stare

Sisteme de gestiune a bazelor de date

În timp ce o bază de date reprezintă o colecție structurată de date, aplicațiile care utilizează informațiile conținute în bazele de date sunt de obicei create pe baza unor instrumente furnizate de un mediu de dezvoltare a bazelor de date. Aceste medii de dezvoltare diferă de mediile tradiționale de programare prin faptul că oferă instrumente specifice care permit utilizatorilor să gestioneze cu ușurință datele, fără a se preocupa de detaliile de nivel coborât asociate în mod normal programării tradiționale (cum ar fi gestionarea memoriei etc.).



Un astfel de mediu de dezvoltare a bazelor de date face parte dintr-un așa-numit Sistem de Gestiune a Bazelor de Date (SGBD).

Un SGBD este o colecție integrată de **instrumente** pentru:

- crearea unei baze de date și specificare structurii acesteia;
- interogarea și modificarea eficientă a datelor;
- securizarea datelor;
- controlul accesului la date de către *mai mulți* utilizatori la *un moment dat*;

Când este util să folosim baze de date? Atunci când nevoile aplicației noastre implică:

- Persistență,
- Cantitate mare de date,
- Date structurate,
- Acces distribuit și concurrent la date,
- Integritate,
- Securitate,
- Partajarea datelor cu alte aplicații.

Când NU utilizăm baze de date?

- Investiția inițială e prea mare,
- Prea mult efort,
- Dezvoltăm o aplicație foarte simplă, bine-definită și care nu presupune modificări ulterioare,
- Nu este necesar accesul mai multor utilizatori la date.

Alternativa: fișiere text.

2. Modelul relațional

Modelul relațional este un model de date utilizat pe scară largă în bazele de date datorită a două caracteristici esențiale: este simplu și elegant.

Simplitatea constă în structurile de date în formă tabelară.

Tabelele sunt simplu de înțeles, sunt utile în modelarea multor situații/entități din lumea reală și conduc la construirea de interogări și limbaje de interogare de o complexitate redusă.

Eleganța este dată de ușurința utilizării matematicii în descrierea și reprezentarea înregistrărilor și a colecțiilor de înregistrări în sub formă de *relații*.

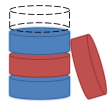
Relațiile pot fi modelate formal, permit utilizarea de limbaje de interogare formale și au proprietăți ce pot fi modelate și demonstrate matematic.

Definiție relație. Un **domeniu** este o mulțime de valori scalare (adică limitate la tipuri atomice - întreg, string, boolean, data calendaristică). O **relație** sau o **structură de relație** **R** este o listă de **nume de atribute** $[A_1, A_2, \dots, A_n]$, având $D_i = \text{Dom}(A_i)$ – domeniul lui A_i , $i=1..n$.

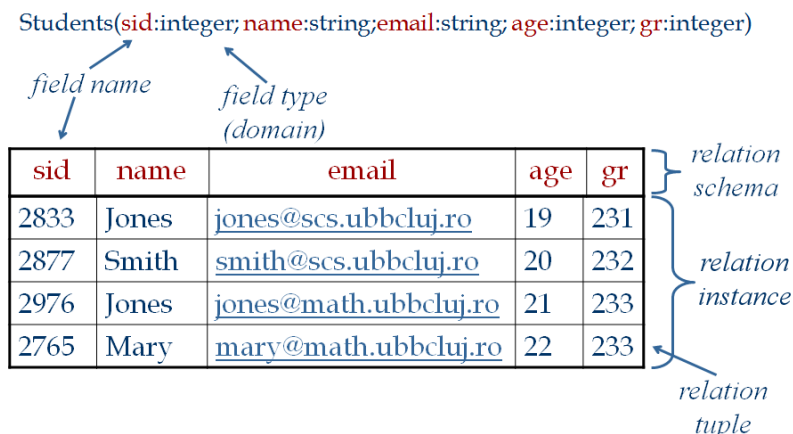
Instanța unei relații ($[R]$) este o submulțime a mulțimii $D_1 \times D_2 \times \dots \times D_n$

Grad (aritate) = numărul tuturor atributelor din structura unei relații

Tuplu = un element al instanței unei relații, o înregistrare. Toate tuplele unei relații sunt distincte!



Cardinalitate = numărul tuplelor unei relații



cardinality = 4, degree = 5, all rows are distinct!

Figura 2.1. Exemplu de relație

Deseori, în vorbirea curentă, dăm același înțeles unor concepte diferite ca:

relație, structură și instanță
instanță și tabelă
atribut, câmp și coloană
tuplu, înregistrare și linie

O **bază de date** este o mulțime de relații. **Structura** unei baze de date este mulțimea structurilor relațiilor bazei de date respective. **Instanța** unei baze de date este mulțimea instanțelor relațiilor bazei de date.

Constrângeri de integritate

Constrângerile de integritate (CI) sunt condiții ce trebuie să fie îndeplinite de către *orice* instanță a bazei de date. CI sunt specificate la momentul definirii structurii unei relații și sunt verificate la modificarea conținutului relației.

Spunem despre o instanță a unei relații că este *legală* dacă satisface toate CI specificate.

Exemple de constrângeri de integritate:

Students(sid:string, name:string, email:string, age:integer, gr:integer)

- Constrângere de domeniu (atributul *gr* poate lua doar valori întregi): *gr:integer*
- Constrângere de interval: $18 \leq \text{age} \leq 70$

Pentru tabelul TestResults (*sid* : string, *TotalQuestions* : integer, *NotAnswered* : integer, *CorrectAnswers* : integer, *WrongAnswers* : integer)

TotalQuestions = *NotAnswered* + *CorrectAnswers* + *WrongAnswers* nu reprezintă o constrângere de integritate!

Definiție cheie primară. O mulțime de atribute reprezintă o **cheie** pentru o relație dacă:

1. Nu există două tuple care au aceleași valori pentru toate atributele
și
2. Acest lucru nu este adevărat pentru nici o submulțime a cheii



Dacă a 2-a afirmație este falsă avem definiție unei **super chei**.

Dacă există mai mult de o cheie pentru o relație acestea se numesc **chei candidat**.

Întotdeauna una dintre cheile candidat este selectată ca **cheie primară**.

Definiție cheie străină (externă). O **cheie străină (externă)** este o mulțime de câmpuri a unei relații utilizate pentru a `referi` un tuplu al unei alte relații (un fel de `pointer logic`). Aceasta trebuie să corespundă cheii primare din a doua relație.

Definiți integritate referențială. **Integritate referențială** reprezintă constrângerea prin care nu este permisă atribuirea de valori unei chei străinw dacă acestea nu se regăsesc în tabela referită de cheia străină.

În cazul tabelelor *Students* și *Enrolled* (figura 2.2) *sid* din *Enrolled* este o cheie străină ce referă o înregistrări din tabela *Students*. Pentru a se respecta integritatea referențială, dacă se încearcă inserarea unei înregistrări în *Enrolled* ce referă un student inexistent, sistemul de gestiune a bazei de date va respinge această înregistrare.

În același timp, dacă o înregistrare din *Students* este ștearsă dar ea este referită din *Enrolled*, atunci se poate opta pentru una dintre următoarele abordări:

- se șterg toate înregistrările corespunzătoare din *Enrolled*;
- nu se permite ștergerea înregistrării referite din *Students*;
- *sid* din *Enrolled* va avea asignată o valoare implicită;
- *sid* din *Enrolled* va avea asignată valoarea *null*.

Aceleași 4 abordări sunt valabile și în cazul în care se modifică valorile cheii primare ale înregistrărilor din *Students*.

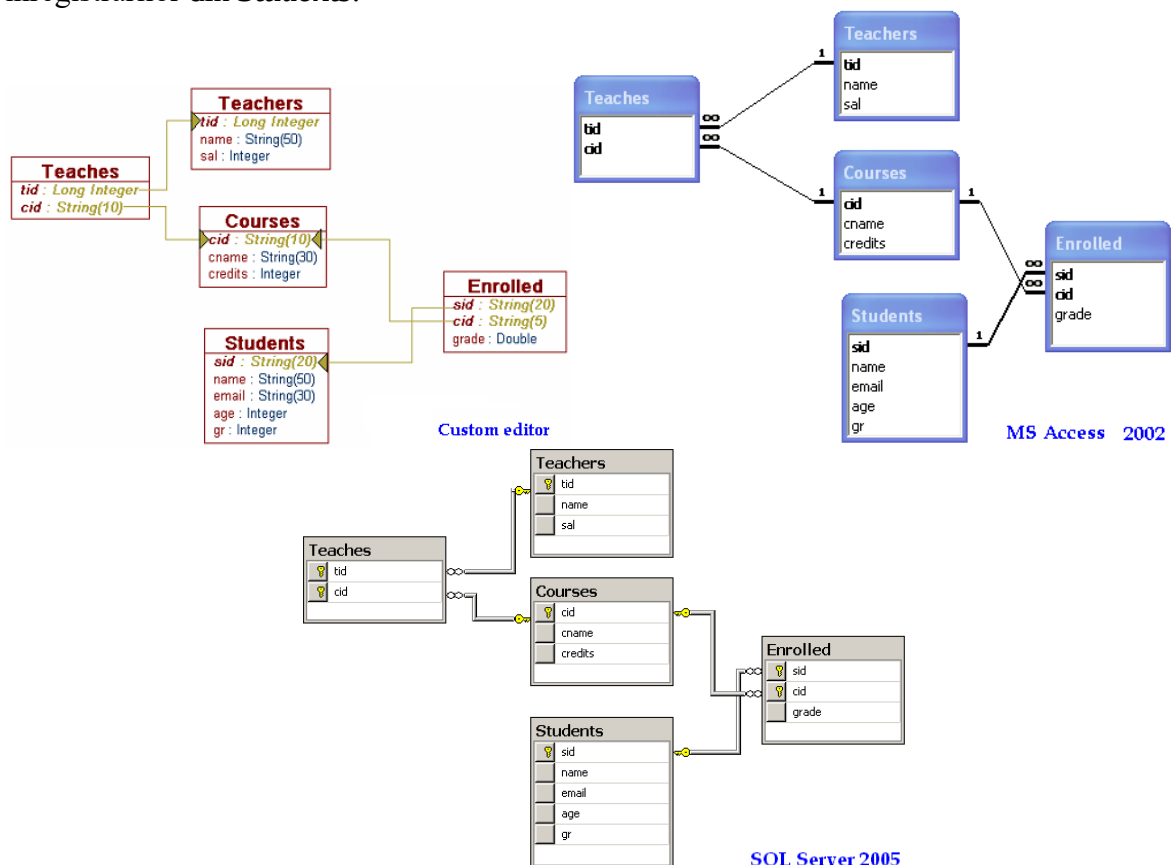
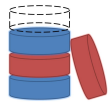


Figura 2.2. Exemple de reprezentări grafice ale tabelor, câmpurilor, cheilor primare și cheilor străine în diverse medii

Cheile primare și străine sunt cele mai comune constrângeri de integritate (CI). CI se bazează pe semantica entităților din lumea reală / conceptuală ce sunt modelate cu ajutorul relațiilor



din baza de date. Putem verifica dacă o CI este violată de instanța unei tabele, însă nu vom putea deduce dacă o CI este adevărată doar consultând o instanță a unei tabele. O CI se referă la *toate instanțele posibile* ale unei tabele particulare!

Nivele de abstractizare

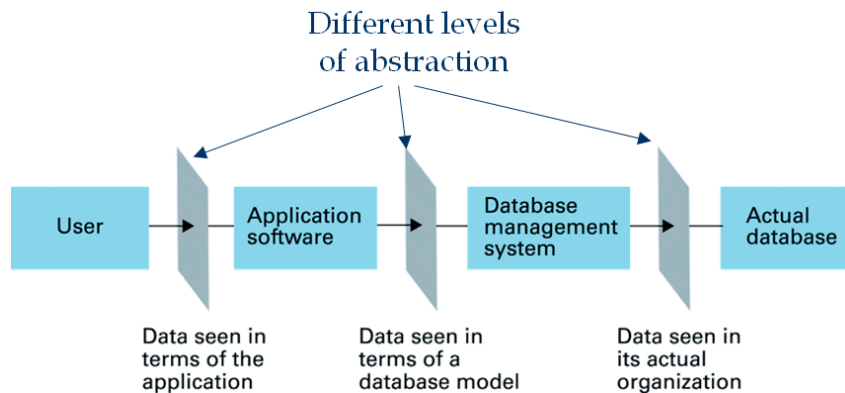


Figura 2.3. Nivelele de abstractizare ale implementării unei baze de date:

Structurile externe (*view*-urile) descriu modul în care utilizatorii văd datele
Structura conceptuală se referă la modelul logic compus din relații, attribute, etc
Structura fizică conține fișierele de date și indecșii utilizați
Exemplu: baza de date **Faculty**
Structura conceptuală:
Students(sid:string, name:string, email:string, age:integer, gr:integer)
Courses(cid: string, cname: string, credits:integer)
Exams(sid:string, cid:string, grade:integer)
Structura fizică:
Relații memorate ca fișiere binare
Fișier index pentru primul atribut din **Students**.
Structura externă (*View*):
Course_info(cid:string, enrollment:integer)

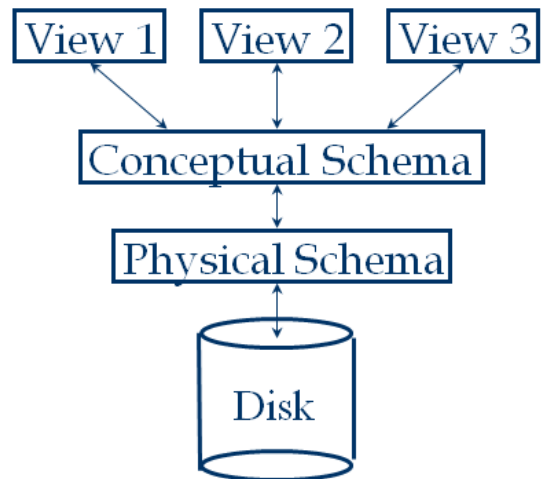


Figura 2.4.

Unul dintre beneficiile importante ale utilizării unui SGBD este separarea logicii aplicațiilor de modul în care datele sunt structurate și stocate.

Independență logică: protecție față de modificările structurii *logice* a datelor.

Independență fizică: protecție față de modificările structurii *fizice* a datelor.

3. Interogări în SGBD-urile relaționale

Se pot enunța mai multe posibile întrebări legate de conținutul bazei de date **Faculty**:



- “Care este numele stui studentului având sid egal cu 2833”?
- “Care este salariul profesorului care predă cursul cu codul Alg100”?
- “Câți studenți sunt înscriși la cursul Alg100”?

Astfel de întrebări, ce implică date stocate într-un SGBD poartă numele de **interogări**. Un SGBD furnizează un limbaj specializat, numit **limbaj de interogare**, ce permite definirea de interogări. Fiecare limbaj de interogare este compus din:

- **Limbaj de definire a datelor (Data Definition Language - DDL)**
Utilizat în definirea structurilor conceptuale;
Include un limbaj de definire a constrângerilor (*constraint definition language - CDL*) pentru descrierea condițiilor pe care trebuie să le satisfacă instanțele bazei de date;
Include un limbaj de definire a modului de stocare (*storage definition language - SDL*) ce permite influențarea arhitecturii structurii fizice (în anumite SGBD-uri);
- **Limbaj de manipulare a datelor (Data Manipulation Language - DML)**
Utilizat pentru descrierea operațiilor aplicate asupra instanței unei baze de date
Exista atât limbaje procedurale (orientate pe *cum*) cât și declarative (*ce*).

Limbaje de interogare pentru baze de date relaționale:

SQL (Structured Query Language):

SELECT *name* FROM *Students* WHERE *age* > 20

Algebră relațională:

$\pi_{name}(\sigma_{age > 20}(Students))$

Domain Calculus:

$\{ \langle X \rangle \mid \exists V \exists Y \exists Z \exists T : Students(V, X, Y, Z, T) \wedge Z > 20 \}$

T-uple Calculus:

$\{ X \mid \exists Y : Y \in Students \wedge Y.age > 20 \wedge X.name = Y.name \}$

Limbajul SQL

Unul dintre avantajele modelului relațional este acela că oferă suport pentru interogări simple dar puternice ale datelor. Interogările pot fi scrise intuitiv, într-un limbaj asemănător cu limbajul natural, iar DBMS-ul este responsabil de evaluare eficientă a acestora.

Structured Query Language (SQL) a devenit limbajul de interogare standard de-facto pentru interogarea bazelor de date relaționale. A fost dezvoltat de IBM (*System R*) în 1970, și a cunoscut diverse extensii ulterioare, trecând prin mai multe etape de standardizare.

Standarde SQL:

SQL-86

SQL-89 (minor revision)

SQL-92 (major revision) - 1,120 pagini

SQL-99 (major extensions) - 2,084 pagini

SQL-2003 (s-a introdus XML ca secțiune nouă) - 3,606 pagini

SQL-2008

SQL- 2011

Sub-limbaje SQL:

- Data-definition language (DDL):
Creare / eliminare / modificare *relații* și *view-uri*.



Definire *constrângeri de integritate* (CI).

- Data-manipulation language (DML)
Permite utilizatorilor să definească interogări
Inserare / ștergere / modificare înregistrări.
- Access Control:
Permite gestionarea drepturilor de acces și manipulare a tabelor și a conținutului lor.

Interogare SQL simplă:

```
SELECT [DISTINCT] target-list  
FROM relation-list  
WHERE qualification
```

- *relation-list* - lista de nume de relații/tabele.
- *target-list* - listă de attribute ale relațiilor din *relation-list*
- *qualification* - comparații logice (Attr op const sau Attr1 op Attr2, unde op unul dintre operatorii <, >, =, ≤, ≥, ≠) combinate cu operatorii logici AND, OR sau NOT.
- *DISTINCT* (optional) - indică faptul că rezultatul final nu conține duplicate.

Evaluarea conceptuală a unei interogări SELECT simple:

- Calcul produs cartezian al tabelor din *relation-list*.
- Filtrare înregistrări ce nu verifică *qualifications*.
- Ștergere attribute ce nu aparțin *target-list*.
- Dacă **DISTINCT** e prezent, se elimină înregistrările duplicate.

4. Dependente funcționale

Proiectare bună / defectuoasă

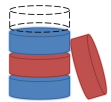
Datele reprezentate prin diverse structuri au în general constrângeri relativ la valorile pe care le pot primi attributele. Aceste constrângeri pot fi utilizate pentru a determina dacă o structură este corect definită/proiectată sau nu.

Exemplu: Să considerăm relația *MovieList* având următoarea instanță:

<i>Title</i>	<i>Director</i>	<i>Cinema</i>	<i>Phone</i>	<i>Time</i>
The Hobbit	Jackson	Cinema City	441111	11:30
The Lord of the Rings 3	Jackson	Cinema City	441111	14:30
Adventures of Tintin	Spielberg	Odeon	442222	11:00
The Lord of the Rings 3	Jackson	Odeon	442222	14:00
War Horse	Spielberg	Odeon	442222	16:30

Figura 3.1 Instanța relației *MovieList*

Datele stocate în această relație trebuie să respecte următoarele constrângeri:



- Fiecare film are un regizor (*director*);
- Fiecare cinematograful are un singur număr de telefon;
- Fiecare cinematograful începe proiecția unui singur film al un moment dat.

Problemele comune care apar în cazul unei proiectări defectuoase sunt:

- **Anomalie de inserare:** Nu putem insera informații despre un nou film dacă nu cunoaștem cel puțin un cinematograful în care acesta este proiectat
- **Anomalie de ștergere:** Dacă eliminăm toate filmele regizate de Peter Jackson, vom pierde informațiile despre cinematograful *Cinema City*
- **Anomalie de modificare:** Dacă numărul de telefon al unui cinematograful se schimbă, trebuie să propagăm această modificare în toate înregistrările unde apare acest cinematograful, în caz contrar rămânând informații inconsistente în baza de date

În general, putem rafina o structură defectuoasă prin *decompunerea* sa în mai multe structuri “bune”.

Movies

<i>Title</i>	<i>Director</i>
The Hobbit	Jackson
The Lord of the Rings 3	Jackson
Adventures of Tintin	Spielberg
War Horse	Spielberg

Cinema

<i>Cinema</i>	<i>Phone</i>
Cinema City	441111
Odeon	442222

Screens

<i>Cinema</i>	<i>Time</i>	<i>Title</i>
Cinema City	11:30	The Hobbit
Cinema City	14:30	The Lord of the Rings 3
Odeon	11:00	Adventures of Tintin
Odeon	14:00	The Lord of the Rings 3
Odeon	16:30	War Horse

Figura 3.2 Descompunerea relației *MovieList*

Rezultatul rafinării structurii prezentat în figura 3.2 permite:

- Inserarea de noi filme (în *Movies*) fără a cunoaște detalii legate de proiectarea acestora
- Eliminarea filmelor fără a pierde informația despre cinematografe
- Modificarea unei singure înregistrări pentru a actualiza numărul de telefon al unui cinematograful.

E important să putem răspunde la următoarele două întrebări:

- Cum determinăm dacă o structură este “*bună*” sau “*defectuoasă*”?
- Cum transformăm o structură *defectuoasă* într-una *bună*?



Teoria bazată pe *dependențe funcționale* furnizează o abordare sistematică a întrebărilor de mai sus. Această teorie a fost introdusă de E.F. Codd în lucrarea: “A relational model for large shared data banks”, Com. of the ACM, 13(6), 1970, pp.377-387.

Dependențe funcționale

Dependențele funcționale (DF) sunt constrângeri ale structurilor (tabelelor) unei baze de date care specifică faptul că valoarea unei anumite mulțimi de attribute determină în mod unic valoarea unei alte mulțimi de attribute.

Fie α și β două submulțimi de attribute ale relației R . Vom nota prin:

$$\alpha \rightarrow \beta$$

faptul că α determină în mod funcțional β (sau β depinde funcțional de α)

În exemplul precedent (relația *MovieList*) putem identifica următoarele dependențe funcționale:

1. Title $\rightarrow$ Director
2. Cinema $\rightarrow$ Phone
3. Cinema, Time $\rightarrow$ Title

Definiție dependență funcțională. Dependența funcțională $\alpha \rightarrow \beta$ este satisfăcută de R dacă și numai dacă pentru *orice* instanță a lui R , oricare două tuple t_1 și t_2 pentru care valorile lui α sunt identice vor avea de asemenea valori identice pentru β .

Acest lucru mai poate fi scris astfel:

$$\pi_{\alpha}(t_1) = \pi_{\alpha}(t_2) \Rightarrow \pi_{\beta}(t_1) = \pi_{\beta}(t_2),$$

Notă: $\pi_{\alpha}(t)$ reprezintă proiecția pe attributele α aplicată tuplei t (v. curs 6)

Fie r instanța unei relații R

Spunem că r **satisfacă DF** $\alpha \rightarrow \beta$ dacă pentru orice pereche de tuple t_1 și t_2 în r astfel încât $\pi_{\alpha}(t_1) = \pi_{\alpha}(t_2)$, este de asemenea adevărat că $\pi_{\beta}(t_1) = \pi_{\beta}(t_2)$. Deci, o **DF f este satisfăcută pe R** dacă și numai dacă orice instanță r a lui R satisfacă f

r **nu respectă** o DF f dacă r nu satisfacă f . Spunem că r este o **instanță legală a lui R** dacă r satisfacă toate dependențele funcționale definite pentru R .

O DF $\alpha \rightarrow \beta$ se numește **trivială** dacă $\alpha \supseteq \beta$

Exemplu. Fie relația **Movie**(Title, Director, Composer) și r o instanță legală a lui **Movie** prezentată mai jos:



<i>Title</i>	<i>Director</i>	<i>Composer</i>
Schindler's List	Spielberg	Williams
Saving Private Ryan	Spielberg	Williams
North by Northwest	Hitchcock	Herrmann
Angela's Ashes	Parker	Williams
Vertigo	Hitchcock	Herrmann

Figura 3.3. Instanță a relației **Movie**

Dependența funcțională $composer \rightarrow director$ nu este respectată de relația **Movie**. În același timp, r satisface DF $director \rightarrow composer$, dar de aici nu putem trage concluzia că $director \rightarrow composer$ este satisfăcută pe întreaga relație **Movie**!

Concluzie: pe baza unor simple instanțe legale ale lui R putem determina care sunt DF ce nu sunt satisfăcute de R , dar nu putem deduce care sunt DF netriviale care sunt respectate de toate instanțele lui R .

Problema implicației: Având dată o mulțime de dependențe funcționale F (care sunt respectate de R) și o dependența funcțională oarecare f , putem deduce dacă f este de asemenea respectată de R ? Spunem că F **implică logic** (sau doar **implică**) f , și notăm prin $F \Rightarrow f$, dacă fiecare instanță r a relației R ce satisface DF din F satisface de asemenea și pe f .

Exemplu: În **MovieList**, avem următoarea mulțime predefinită de dependențe funcționale:

$$F = \{ \text{Title} \rightarrow \text{Director} \\ \text{Cinema} \rightarrow \text{Phone} \\ \text{Cinema, Time} \rightarrow \text{Title} \}$$

Dependențele $\text{Cinema, Time} \rightarrow \text{Director}$ sau $\text{Time} \rightarrow \text{Director}$ sunt de asemenea respectate?

Fie F & G două mulțimi de dependențe funcționale. Vom nota prin $F \Rightarrow G$ (F implică G) dacă $F \Rightarrow g$ pentru oricare $g \in G$.

Definiție închidere mulțime de dependențe funcționale. Includerea lui F (notată prin F^+) este mulțimea tuturor dependențelor funcționale implicate de F și anume,

$$F^+ = \{f \mid F \Rightarrow f\}$$

Spunem că două mulțimi de dependențe funcționale, F și G , sunt **echivalente** (notat prin $F \equiv G$) dacă $F^+ = G^+$ (adică, $F \Rightarrow G$ și $G \Rightarrow F$)

Axiomele dependențelor funcționale reprezintă o colecție de reguli formale de derivare a dependențelor funcționale dintr-un set inițial de dependențe funcționale.

Axiomele lui Armstrong: Fie $\alpha, \beta, \gamma \subseteq R$



Reflexivitate: Dacă $\beta \subseteq \alpha$, atunci $\alpha \rightarrow \beta$

Augmentare: Dacă $\alpha \rightarrow \beta$, atunci $\alpha\gamma \rightarrow \beta\gamma$

Tranzitivitate: Dacă $\alpha \rightarrow \beta$ și $\beta \rightarrow \gamma$, atunci $\alpha \rightarrow \gamma$

Axiomele lui Armstrong sunt *necesare* și *suficiente* pentru a determina DF implicate de F

Necesare: Orice FD derivată este implicată de F

Complete: Toate DF din F^+ pot fi derivate

Problemă: Considerăm $R(A, B, C, D, E)$ având 3 dependențe funcționale:

$$F = \{A \rightarrow C; B \rightarrow C; CD \rightarrow E\}.$$

Arătați că $F \Rightarrow AD \rightarrow E$

Soluție:

1. $A \rightarrow C$ (dată)
2. $AD \rightarrow CD$ (augmentare cu (1))
3. $CD \rightarrow E$ (dată)
4. $AD \rightarrow E$ (tranzitivitate cu (2) și (3))

Reguli de inferență adiționale

Reuniunea: Dacă $\alpha \rightarrow \beta$ și $\alpha \rightarrow \gamma$, atunci $\alpha \rightarrow \beta\gamma$

Decompunerea: Dacă $\alpha \rightarrow \beta$, atunci $\alpha \rightarrow \beta'$ pentru orice $\beta' \subseteq \beta$

Superchei, chei & attribute prime

O mulțime de attribute α reprezintă o supercheie a relației R (având mulțimea de DF F) dacă

$$F \Rightarrow \alpha \rightarrow R.$$

O mulțime de attribute α o cheie a relației R dacă

- (1) α este o supercheie, și
- (2) nici o submulțime a lui α nu este supercheie
(adică, pentru fiecare $\beta \subset \alpha$, $\beta \rightarrow R \notin F^+$)

Un atribut $A \in R$ se numește atribut prim dacă A face parte dintr-o cheie a lui R; în caz contrar, A se numește atribut neprim.

Exemplu: Considerăm din nou relația **MovieList** (Title, Director, Cinema, Phone, Time) cu mulțimea de dependențe funcționale:

- (1) Cinema, Time $\rightarrow$ Title
- (2) Cinema $\rightarrow$ Phone
- (3) Title $\rightarrow$ Director

{Cinema, Time} este singura cheie a relației **MovieList**.



Cinema și *Time* sunt singurele atribute prime din **MovieList**.

Orice mulțime ce include $\{Cinema; Time\}$ este o supercheie a lui **MovieList**.

Închiderea atributelor

Identificarea tuturor dependențelor funcționale din F^+ pornind de la o mulțime F de DF nu este eficientă deoarece dimensiunea lui F^+ crește exponențial!

Este, astfel, mult mai eficient să se calculeze închiderea unei mulțimi de atribute

Definiție închiderea unui atribut. Fie $\alpha \subseteq R$ și F o mulțime de DF satisfăcute pe R .

Închiderea lui α (cu respectarea mulțimii F de DF), notată cu α^+ , este mulțimea de atribute ce sunt determinate funcțional din α pe baza dependențelor funcționale din F ; adică

$$\alpha^+ = \{A \in R \mid F \Rightarrow \alpha \rightarrow A\}$$

Se observă că $F \Rightarrow \alpha \rightarrow \beta$ dacă și numai dacă $\beta \subseteq \alpha^+$ (cu respectarea DF din F)

Algoritm de determinarea a închiderii unui atribut/unei mulțimi de atribute:

Input: α, F

Output: α^+ (w.r.t. F)

Se calculează secvența de mulțimi de atribute $\alpha_0, \alpha_1, \dots, \alpha_k, \alpha_{k+1}$ astfel:

$$\alpha_0 = \alpha$$

$$\alpha_{i+1} = \alpha_i \cup \gamma \text{ astfel încât exista o FD}$$

$$\beta \rightarrow \gamma \in F \text{ și } \beta \subseteq \alpha_i$$

Algoritmul se termina atunci când $\alpha_{k+1} = \alpha_k$ pentru un anumit k

Return α_k

Problemă: Fiind dată $F = \{A \rightarrow C; B \rightarrow C; CD \rightarrow E\}$, arătați că $F \Rightarrow AD \rightarrow E$.

Soluție

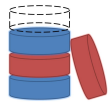
i	α_i	FD used
0	AD	given input
1	ACD	$A \rightarrow C$
2	ACDE	$CD \rightarrow E$
3	ACDE	none

Figura 3.4. Determinarea închiderii atributului AD

Deci $AD^+ = ACDE$. Deoarece $E \in AD^+$, atunci rezultă că $F \Rightarrow AD \rightarrow E$

5. Descompunerea relațiilor

Descompunerea unei relații R este un set de (sub)relații $\{R_1, R_2, \dots, R_n\}$ astfel încât fiecare $R_i \subseteq R$ și $R = \bigcup R_i$. Dacă r este o relație din R , atunci r se descompune în $\{r_1, r_2, \dots, r_n\}$, unde



fiecare

$$r_i = \pi_{R_i}(r)$$

Exemplu:

{ (Cinema, Time, Title),
(Title, Director),
(Cinema, Phone)}

Este o descompunere a relației **MovieList**(Title, Director, Cinema, Phone, Time)

Proprietățile descompunerii relațiilor:

1. Descompunerea trebuie să păstreze informațiile
 - Datele din relația originală $\equiv$ Data din relațiile descompunerii
 - Crucial for păstrarea consistenței datelor!
2. Descompunerea trebuie să respecte toate DF
 - Dependențele funcționale din relația originală $\equiv$ reuniunea dependențelor funcționale din relațiile descompunerii
 - Facilitează verificarea violărilor DF

Descompunere cu joncțiune fără pierderi

Este important ca descompunerea unei relații să păstreze informațiile; ceea ce înseamnă că putem reconstrui o instanță r prin joncțiunea proiecțiilor sale $\{r_1, r_2, \dots, r_n\}$. De notat că dacă $\{R_1, R_2, \dots, R_n\}$ este o descompunere a lui R , atunci pentru orice instanță r a lui R , este întotdeauna adevărat că

$$r \subseteq \pi_{R_1}(r) \otimes \pi_{R_2}(r) \otimes \dots \otimes \pi_{R_n}(r)$$

Notă: $\otimes$ reprezintă operatorul algebric join natural (v cursul 6)

O descompunere a lui R (cu mulțimea de dependențe funcționale F) în $\{R_1, R_2, \dots, R_n\}$ este o descompunere cu joncțiuni fără pierderi cu respectarea mulțimii F dacă

$$\pi_{R_1}(r) \otimes \pi_{R_2}(r) \otimes \dots \otimes \pi_{R_n}(r) = r$$

Pentru fiecare instanță r a lui R ce satisface F .

Exemplu. Se consideră descompunerea relației $R(A,B,C)$ în $\{R_1(AC), R_2(BC)\}$ prezentată în figura 3.5

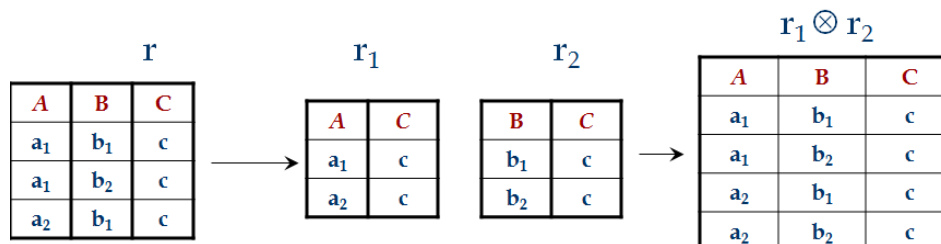
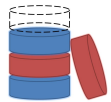


Figura 3.5. O descompunere a instanței r a relației $R(A, B, C)$

Observăm din figura 3.5 că $r \subset r_1 \otimes r_2$, deci descompunerea propusă nu este cu joncțiuni fără pierderi.



Cum determinăm dacă $\{R_1, R_2\}$ este o descompunere cu joncțiuni fără pierderi a lui R ?

Teoremă: Descompunerea lui R (cu mulțimea F de DF) în $\{R_1, R_2\}$ este cu joncțiuni fără pierderi cu respectarea mulțimii F dacă și numai dacă:

$$F \Rightarrow R_1 \cap R_2 \rightarrow R_1 \text{ sau } F \Rightarrow R_1 \cap R_2 \rightarrow R_2$$

Cum descompunem R în $\{R_1, R_2\}$ astfel încât aceasta e cu joncțiuni fără pierderi?

Corolar: Dacă $\alpha \rightarrow \beta$ este satisfăcută pe R și $\alpha \cap \beta = \emptyset$, atunci descompunerea lui R în $\{R - \beta, \alpha\beta\}$ este o descompunere cu joncțiuni fără pierderi.

Exemple. Se consideră $R(A,B,C)$ cu mulțimea de DF $F = \{A \rightarrow B\}$

Descompunerea $\{AB, AC\}$ este cu joncțiuni fără pierderi deoarece $AB \cap AC = A$ și $A \rightarrow AB$

Descompunerea $\{AB, BC\}$ nu este cu joncțiuni fără pierderi deoarece $AB \cap BC = B$ și nici $B \rightarrow AB$ sau $B \rightarrow BC$ nu sunt satisfăcute de R .

Teoremă: Dacă $\{R_1, R_2\}$ este o descompunere cu joncțiuni fără pierderi a lui R , și dacă $\{R_{1,1}, R_{1,2}\}$ este o descompunere cu joncțiuni fără pierderi a lui R_1 , atunci $\{R_{1,1}, R_{1,2}, R_2\}$ este o descompunere cu joncțiuni fără pierderi a lui R (figura 3.6)

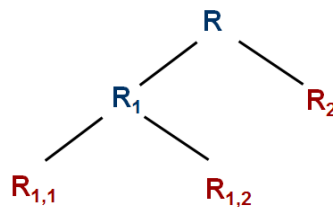


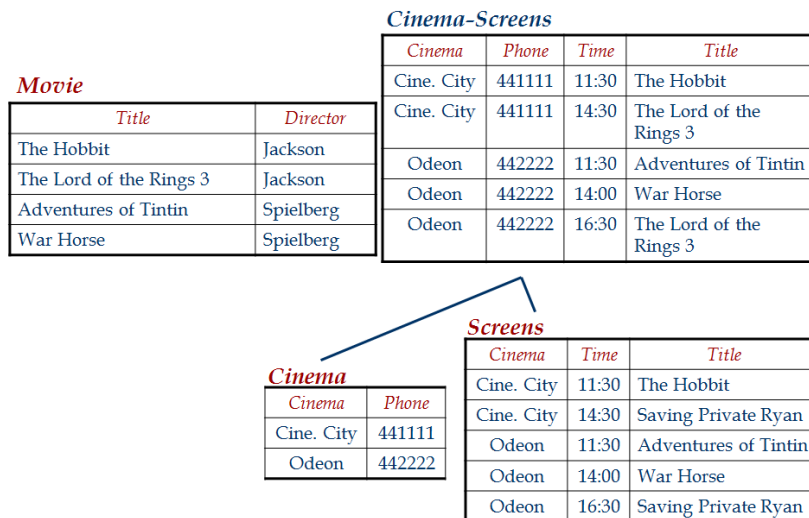
Figura 3.6. Descompunerea relației R în $\{R_{1,1}, R_{1,2}, R_2\}$ conform enunțului teoremei.

<i>MovieList</i>				
<i>Title</i>	<i>Director</i>	<i>Cinema</i>	<i>Phone</i>	<i>Time</i>
The Hobbit	Jackson	Cinema City	441111	11:30
The Lord of the Rings 3	Jackson	Cinema City	441111	14:30
Adventures of Tintin	Spielberg	Odeon	442222	11:30
War Horse	Spielberg	Odeon	442222	14:00
The Lord of the Rings 3	Jackson	Odeon	442222	16:30

<i>Movie</i>		<i>Cinema-Screens</i>			
<i>Title</i>	<i>Director</i>	<i>Cinema</i>	<i>Phone</i>	<i>Time</i>	<i>Title</i>
The Hobbit	Jackson	Cine. City	441111	11:30	The Hobbit
The Lord of the Rings 3	Jackson	Cine. City	441111	14:30	The Lord of the Rings 3
Adventures of Tintin	Spielberg	Odeon	442222	11:30	Adventures of Tintin
War Horse	Spielberg	Odeon	442222	14:00	War Horse
		Odeon	442222	16:30	The Lord of the Rings 3

Figura 3.7. Primul pas al descompunerii relației **MovieList** pe baza DF

$$Title \rightarrow Director$$

Figura 3.8 Ultimul pas al descompunerii **MovieList**, pe baza DF $Cinema \rightarrow Phone$

Păstrarea dependențelor funcționale

Numim proiecția lui F pe α (notată prin F_α) mulțimea dependențelor funcționale din F^+ ce implică doar atributele din α ; adică $F_\alpha = \{ \beta \rightarrow \gamma \in F^+ \mid \beta\gamma \subseteq \alpha \}$

Algoritmul de determinare a proiecției unei mulțimi de dependențe funcționale:

```

Input:  $\alpha, F$ 
Output:  $F_\alpha$ 
result =  $\emptyset$ ;
for each  $\beta \subseteq \alpha$  do
     $T = \beta^+$  (w.r.t.  $F$ )
    result = result  $\cup \{ \beta \rightarrow T \cap \alpha \}$ 
return result

```

Definiție. Descompunerea $\{R_1, R_2, \dots, R_n\}$ relației R păstrează dependențele dacă

$(F_{R_1} \cup F_{R_2} \cup \dots \cup F_{R_n})$ și F sunt echivalente, adică:

$(F_{R_1} \cup F_{R_2} \cup \dots \cup F_{R_n}) \Rightarrow F$ și $F \Rightarrow (F_{R_1} \cup F_{R_2} \cup \dots \cup F_{R_n})$

6. Forme normale

Redundanța este cauza principală a majorității problemelor legate de structura bazelor de date relaționale:

- spațiul utilizat,
- anomalii de inserare / stergere / actualizare

Dependențele funcționale pot fi utilizate pentru identificarea problemelor de proiectare și sugerează posibile îmbunătățiri.



De exemplu, fie relația R cu 3 atribute, ABC. Avem următoarele cazuri

- Nici o dependență funcțională satisfăcută implică absența redundanțelor.
- Existența dependenței $A \rightarrow B$ implică faptul că atunci când mai multe înregistrări au aceeași valoare pentru A, avem valori identice pentru B.

Tehnica de rafinare a structurii pentru minimizarea redundanțelor este *descompunerea*. Descompunerea trebuie folosită cu "măsură" și de aceea trebuie să ne asigurăm că este într-adevăr necesară o rafinare, că există motive de decompunere a unei relații și să identificăm eventualele probleme ce pot să apară prin descompunere.

Forme Normale

Dacă o relație se află într-o *formă normală* particulară avem certitudinea că anumite categorii de probleme sunt eliminate/minimizate $\rightarrow$ ne ajută să decidem dacă descompunerea unei relații este necesară sau nu.

Formele normale bazate pe DF sunt:

- *prima formă normală (1NF)*,
- *a doua formă normală (2NF)*,
- *a treia formă normală (3NF)*,
- *forma normală Boyce-Codd (BCNF)*.

$$\{BCNF \subseteq 3NF, 3NF \subseteq 2NF, 2NF \subseteq 1NF\}$$

Definiție 1NF. O relație se află în *Prima Formă Normală (1NF)* dacă fiecare atribut al relației poate avea doar valori atomice (deci listele și mulțimile sunt excluse).

Această condiție este implicită conform definiției de model relational.

Definiție Dependență Funcțională Parțială. Spunem că avem o *dependență funcțională parțială* într-o relație atunci când un atribut *ne-cheie* este dependent funcțional de o parte a unei chei candidat a relației (dar nu de întreaga cheie).

Definiție 2NF. O relație se află în *A Doua Formă Normală (2NF)* dacă este 1NF și nu are dependențe parțiale.

Definiție BCNF. O relație R ce satisface dependențele funcționale F se află în *Forma Normală Boyce-Codd (BCNF)* dacă se află în 2NF și pentru toate $\alpha \rightarrow A$ din F^+ :

- $A \in \alpha$ (DF *trivială*), sau
- α conține o cheie a lui R.

Cu aște cuvinte o relație R este în BCNF dacă singurele dependențe funcționale satisfăcute de R sunt cele corespunzătoare constrângerilor de cheie.

Definiție 3NF. O relație R ce satisface dependențele funcționale F se află în *A Treia Formă Normală (3NF)* dacă se află în 2NF și pentru toate $\alpha \rightarrow A$ din F^+

- $A \in \alpha$ (DF *trivială*), sau
- α conține o cheie de-a lui R, sau
- A este un atribut prim.

Observații:



- Dacă R este în BCNF, evident este și în 3NF.
- Dacă R este în 3NF, este posibil ca să apară anumite redundanțe. Este un compromis, utilizat atunci când BCNF nu se poate atinge.
- *Descompunerea cu joncțiune fără pierderi & cu păstrarea dependențelor a relației R într-o mulțime de relații 3NF este întotdeauna posibilă.*

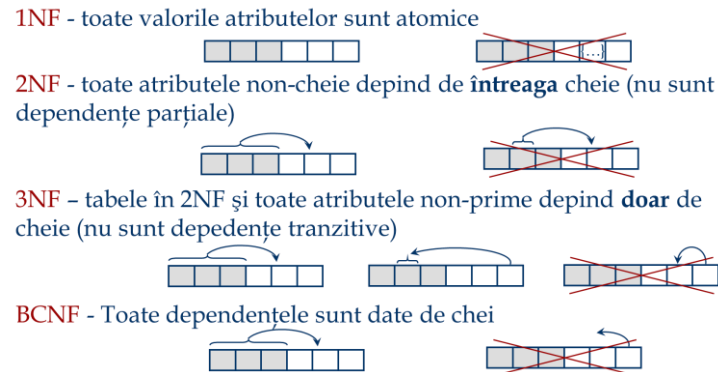


Figura 6.1. Descrierea simplificată a primelor 3 forme normale.

Exemple de nerespectare a formelor normale:

2NF - toate atributele neprime trebuie să depindă de **întreaga** cheie

Exam (**Student**, **Course**, Teacher, Grade) (**Course** → **Teacher**)

3NF - toate atributele neprime trebuie să depindă **doar** de cheie

Dissertation(**Student**, Title, Teacher, Department) (**Teacher** → **Department**)

BCNF - toate DF sunt implicate de cheile candidat

Schedule (**Day**, **Route**, **Bus**, Driver) (**Driver** → **Bus**)

"Strategia" de normalizare:

- prima alegere: BCNF prin descompunere cu joncțiune fără pierderi și păstrarea dependențelor
- a doua alegere: 3NF prin descompunere cu joncțiune fără pierderi și păstrarea dependențelor

Vom alege a doua variantă atunci când dependențele nu pot fi păstrate pentru a obține BCNF.

Descompunerea în BCNF: Fie relația R cu dependențele funcționale F. Dacă $\alpha \rightarrow A$ nu respectă BCNF, descompunem R în R - A și αA .

Aplicarea repetată a acestei idei va conduce la o colecție de relații care

- sunt în BCNF;
- conduc la joncțiune fără pierderi;
- garantează terminarea.

În general, însă, descompunerea în BCNF nu păstrează dependențele.

Exemplu. $R(C, S, Z)$, $\{CS \rightarrow Z, Z \rightarrow C\}$

Exemplu. $R(C, S, J, P, D, Q, V)$ descompus în (S, D, P) , (J, S) și (C, J, D, Q, V) nu păstrează dependențele inițiale $\{JP \rightarrow C, SD \rightarrow P, J \rightarrow S\}$.

Adăugând JPC la mulțimea de relații obținem *descompunere cu păstrarea dependențelor* (dar revenim la... redundanță!)



Descompunerea în 3NF: Evident, procedeul descompunerii din BCNF poate fi utilizat și pentru descompunerea 3NF.

Cum asigurăm păstrarea dependențelor? Dacă $X \rightarrow Y$ nu se păstrează, adăugăm XY .

Problema este că XY e posibil să nu respecte 3NF!

Rafinare: În loc de a utiliza mulțimea inițială F , folosim conceptul de *acoperire minimală* a lui F .

Definiție atribut redundant. Un atribut $A \in \alpha$ e redundant în DF $\alpha \rightarrow B$ dacă

$$(F - \{\alpha \rightarrow B\}) \cup \{\alpha - A \rightarrow B\} \equiv F$$

Pentru a verifica dacă $A \in \alpha$ e redundant în $\alpha \rightarrow B$, calculăm $(\alpha - A)^+$. Apoi $A \in \alpha$ e redundant în $\alpha \rightarrow B$ dacă $B \in (\alpha - A)^+$.

Exercițiu: Care sunt attributele redundante în $AB \rightarrow C$ având:

$$\{AB \rightarrow C, A \rightarrow B, B \rightarrow A\}?$$

Definiție dependență funcțională redundantă. O DF $f \in F$ e redundantă dacă mulțimea de dependențe funcționale $F - \{f\}$ e echivalentă cu F .

În general, vom verifica dacă $\alpha \rightarrow A$ e redundantă în F calculând α^+ pe baza $F - \{\alpha \rightarrow A\}$. Vom avea că $\alpha \rightarrow A$ e redundantă în F dacă $A \in \alpha^+$.

Exercițiu: Care sunt dependențele funcționale redundante în:

$$\{A \rightarrow C, A \rightarrow B, B \rightarrow A, B \rightarrow C, C \rightarrow A\}?$$

Definiție acoperire minimală. O acoperire minimală pentru mulțimea F de dependențe funcționale este o mulțime G de dependențe funcționale pentru care:

1. Fiecare dependență funcțională din G e de forma $\alpha \rightarrow A$
2. Pentru fiecare dependență funcțională $\alpha \rightarrow A$ din G , α nu are attribute redundante
3. Nu sunt dependențe funcționale redundante în G
4. G și F sunt echivalente

Fiecare mulțime de dependențe funcționale are cel puțin o acoperire minimală!

Algoritm de calcul al acoperirii minime pentru o mulțime de dependențe funcționale F :

1. Folosim descompunerea pentru a obține DF cu 1 atribut în partea dreaptă.
2. Se elimină attributele redundante
3. Se elimină dependențele funcționale redundante

Exercițiu: Să se determine o acoperire minimală a mulțimii de dependențe funcționale $F = \{ABCD \rightarrow E, E \rightarrow D, A \rightarrow B, AC \rightarrow D\}$.

Rezolvare: Attributele BD din $ABCD \rightarrow E$ sunt redundante. După eliminare obținem:

$$F = \{AC \rightarrow E, E \rightarrow D, A \rightarrow B, AC \rightarrow D\}$$

$AC \rightarrow D$ este o dependență funcțională redundantă. După eliminare obținem:

$$F = \{AC \rightarrow E, E \rightarrow D, A \rightarrow B\}$$

care este o acoperire minimală

Acoperirile minime nu sunt unice (depind de ordinea de alegere a DF/atr. redundante)



Algoritm de descompunere a un relații ce nu se află în a 3-a formă normală (3NF) în sub-relații ce se află în 3NF:

Input: Schema R with F which is a minimal cover
Output: A dependency-preserving, lossless-join 3NF decomposition of R
Initialize $D = \emptyset$
Apply *union rule* to combine FDs in F with same L.H.S. into a single FD
For each FD $\alpha \rightarrow \beta$ in F do
 Insert the relation schema $\alpha\beta$ into D
 Insert δ into D , where δ is some key of R
Remove redundant relation schema from D as follows:
 delete R_i from D if $R_i \subseteq R_j$, where $R_j \in D$
return D

Exercițiu: Fie $R(A,B,C,D,E)$ cu dependentele functionale:

$$F = \{ABCD \rightarrow E, E \rightarrow D, A \rightarrow B, AC \rightarrow D\}$$

Să se descompună R în relații care se află în 3NF.

Schiță rezolvare:

- Acoperirea minimală a F este $\{AC \rightarrow E, E \rightarrow D, A \rightarrow B\}$
- Unica cheie: AC
- R nu e în 3NF deoarece $A \rightarrow B$ nu respecta 3NF
- descompunerea 3NF a R :
 - Relații pentru fiecare DF: $R_1(A, C, E)$, $R_2(E, D)$, și $R_3(A, B)$
 - Relație pentru cheia lui R : $R_4(A, C)$
 - Eliminare relație redundantă: R_4 (deoarece $R_4 \subseteq R_1$) $\Rightarrow$ descompunerea 3NF este $\{R_1(A, C, E), R_2(E, D), R_3(A, B)\}$

Descompunerea 3NF nu este unică. Aceasta depinde de:

- Alegerea acoperirii minime sau
- Alegerea relatiei redundante care va fi eliminată

Descompunerea este, în general, ultima soluție de rezolvare a problemelor generate de redundanțe & anomalii. Descompunerea excesivă poate conduce la afectarea performanței bazei de date și la obținerea unei structuri a bazei de date nejustificat de complexe.

În același timp uneori, din motive de performanță se practică de-normalizarea datelor/relațiilor.

Definiție dependențe multivaloare. Fie α, β două submulțimi de atribute din R . Dependența multivaloare $\alpha \twoheadrightarrow \beta$ este respectată de R dacă, pentru fiecare instanță validă r a lui R , fiecare valoare α este asociată cu o mulțime de valori pentru β și această mulțime de valori este independentă de valorile altor atribute.

Formal: dacă $\alpha \twoheadrightarrow \beta$ e respectată de R și $\gamma = R - \alpha\beta$, următoarea afirmație e adevărată pentru orice instanță validă r a lui R :

$$\begin{aligned} & t_1, t_2 \in r \text{ și } \pi_\alpha(t_1) = \pi_\alpha(t_2) \Rightarrow \\ & \exists t_3 \in r \text{ a.î. } \pi_{\alpha\beta}(t_1) = \pi_{\alpha\beta}(t_3) \text{ și } \pi_\gamma(t_2) = \pi_\gamma(t_3) \end{aligned}$$



În consecință, pentru t_2 și t_1 se poate deduce că există și $t_4 \in r$ astfel încât:

$$\pi_{\alpha\beta}(t_2) = \pi_{\alpha\beta}(t_4) \text{ și } \pi_{\gamma}(t_1) = \pi_{\gamma}(t_4)$$

Exemplu:

	X	Y	Z
t_1 →	a	b_1	c_1
t_2 →	a	b_2	c_2
t_3 →	a	b_1	c_2
t_4 →	a	b_2	c_1

Figura 6.2. Instanță de relație ce conține dependențe multivaloare

$$\forall t_1, t_2 \in r \text{ și } \pi_x(t_1) = \pi_x(t_2) \Rightarrow$$

$$\exists t_3 \in r \text{ astfel încât}$$

$$\pi_{XY}(t_1) = \pi_{XY}(t_3),$$

$$\pi_z(t_2) = \pi_z(t_3)$$

Reguli adiționale:

- Complementare: $X \twoheadrightarrow Y \Rightarrow X \twoheadrightarrow R - XY$
- Augumentare: $X \twoheadrightarrow Y, Z \subseteq W \Rightarrow WX \twoheadrightarrow YZ$
- Tranzitivitate: $X \twoheadrightarrow Y, Y \twoheadrightarrow Z \Rightarrow X \twoheadrightarrow Z - Y$
- Replicare: $X \rightarrow Y \Rightarrow X \twoheadrightarrow Y$
- Fuzionare: $X \twoheadrightarrow Y, W \cap Y = \emptyset, W \rightarrow Z, Z \subseteq Y \Rightarrow X \rightarrow Z$

Definiție 4NF. Fie R o schemă relațională și F o mulțime de dependențe funcționale și multivaloare pe R .

Spunem că R este în a patra forma normală NF4 dacă, pentru orice dependență multivaloare $X \twoheadrightarrow Y$:

- $Y \subseteq X$ sau
- $XY = R$ sau
- X e super-cheie

Exercițiu:

Relația $R(\text{course}, \text{teacher}, \text{book})$ având cheia primară formată din toate cele trei atribute se află în BCNF dar nu este în 4NF.

course	teacher	book
alg101	Green	Alg Basics
alg101	Green	Alg Theory
alg101	Brown	Alg Basics
alg101	Brown	Alg Theory
logic203	Green	Logic B.
logic203	Green	Logic F.
logic203	Green	Logic intro.

Figura 6.3 Instanța a relației $R(\text{course}, \text{teacher}, \text{book})$

Avem $\text{course} \twoheadrightarrow \text{teacher}$

Relația se poate descompune în: $(\text{Course}, \text{Teacher})$ și $(\text{Course}, \text{Book})$, ambele relații fiind în 4NF (figura 6.4).



course	teacher
alg101	Green
alg101	Brown
logic203	Green

course	book
alg101	Alg Basics
alg101	Alg Theory
logic203	Logic B.
logic203	Logic F.
logic203	Logic intro.

Figura 6.3 Instanțele obținute în urma descompunerii lui R în $R_1(\text{course}, \text{teacher})$ și $R_2(\text{course}, \text{book})$

Definiție dependența join. Spunem ca R satisface dependența *join* $\otimes\{R_1, \dots, R_n\}$ dacă $R_1, R_2, \dots, R_n$ este o descompunere cu joncțiuni fără pierderi a lui R.

O dependență multivaloare $X \twoheadrightarrow Y$ poate fi exprimată ca o dependență join:

$$\otimes\{XY, X(R-Y)\}.$$

Definiție 5NF. O relație R este în 5NF dacă și numai dacă pentru orice dependență *join* a lui R:

- $R_i = R$ pentru un i oarecare, sau
- dependența este implicată de o mulțime de dependențe functionale din R în care partea stângă e o cheie pentru R

7. Algebra relațională

Limbaje de interogare relațională

Limbaj de interogare: Permite manipularea și regăsirea datelor dintr-o bază de date.

Modelul relațional oferă suport pentru limbaje de interogare simple & puternice deoarece are un fundament formal, bazat pe logică, și implică o plajă largă de optimizări.

Limbajele de interogare **nu sunt** limbaje de programare deoarece nu sunt “Turing complete” și nu sunt utilizate pentru calcule complexe. Limbajele de interogare oferă o modalitate simplă și eficientă de acces la mulțimi de date voluminoase.

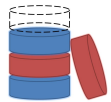
Două limbaje de interogare formează baza pentru limbajele utilizate în practică (ex. SQL):

- Algebra Relațională: Mai operatională, utilă pentru reprezentarea planurilor de execuție.
- Relational Calculus: Permite utilizatorilor să descrie **ce**, și nu **cum** să obțină ceea ce doresc. (Non-operational, declarativ)

Algebra relațională

O interogare se aplică *instanței* unei relații, și rezultatul interogării reprezintă de asemenea o instanță de relație. *Structura* relațiilor ce apar într-o interogare este fixă (dar interogarea se va executa indiferent de instanța relației la un moment dat). Structura *rezultatului* unei interogări este de asemenea fixă și este determinată de definițiile construcțiilor limbajului de interogare.

Notăția utilizată poate fi pozițională sau prin nume. Notăția pozițională este mai utilă în definiții formale, însă utilizarea numelor de câmpuri conduce la interogări mai ușor de citit. Ambele variante sunt utilizate în SQL.



Algebra relațională propune un set de operatori de bază și unul de operatori adiționali care pot fi modelați/descriși folosind operatorii de bază. Operatorii adiționali simplifică în multe cazuri descrierea unei interogări.

Operatorii algebrici de bază sunt:

- Proiecția (π) - Elimină attributele nedorite ale unei relații,
- Selectia (σ) - Selectează o submulțime de tupluri ale unei relații,
- Produsul cartezian ($\times$) - Permite combinarea a două relații,
- Diferența ($-$) - Tuplurile ce aparțin unei relații dar nu aparțin celeilalte,
- Reuniunea ($\cup$) - Tuplurile aparținând ambelor relații.

Operatorii adiționali cei mai importanți sunt: intersecția, join, câțul, redenumirea etc.

Deoarece fiecare operator returnează o relație, operatorii pot fi compuși (algebra este “închisă”).

Proiecția. Fie $L = (a_1, \dots, a_n)$ este o listă de attribute (sau o *lista de coloane*) ale relației R . Operatorul proiecție (notat prin π_L) aplicat unei relații R returnează o relație ce păstrează doar attributele care nu sunt în L (și care apare ca indice al operatorului):

$$\pi_L(R) = \{ t \mid t_1 \in R \wedge t.a_1 = t_1.a_1 \wedge \dots \wedge t.a_n = t_1.a_n \}$$

Exemplu: $\pi_{cid, grade}(\text{Enrolled})$

<i>sid</i>	<i>cid</i>	<i>grade</i>
1234	Alg1	9
1235	Alg1	10
1234	DB1	10
1234	DB2	9
1236	DB1	7
1237	DB2	9
1237	DB1	5
1237	Alg1	10

 $=$

<i>cid</i>	<i>grade</i>
Alg1	9
Alg1	10
DB1	10
DB2	9
DB1	7
DB1	5

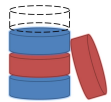
Figura 7.1. Rezultatul aplicării proiecției pe instanța particulară a unei relații

Proiecția $\pi_{cid, grade}(\text{Enrolled})$ este echivalentă cu interogarea SQL `SELECT DISTINCT cid, grade FROM Enrolled` (algebra relațională operează cu mulțimi și prin definiție relațiile nu conțin duplicate).

Selectia. Operatorul selecție (notat cu σ_c) aplicat unei relații R selectează tuplurile acestei relații care verifică o condiție c (numită și *predicat de selecție*).

$$\sigma_c(R) = \{ t \mid t \in R \wedge c \}$$

Exemplu. $\sigma_{grade > 8}(\text{Enrolled}) = \{ t \mid t \in \text{Enrolled} \wedge grade > 8 \}$



$$\sigma_{\text{grade} > 8} \left(\begin{array}{|c|c|c|} \hline \textit{sid} & \textit{cid} & \textit{grade} \\ \hline 1234 & \text{Alg1} & 9 \\ 1235 & \text{Alg1} & 10 \\ 1234 & \text{DB2} & 9 \\ 1236 & \text{DB1} & 7 \\ 1237 & \text{DB1} & 5 \\ 1237 & \text{Alg1} & 6 \\ \hline \end{array} \right) = \begin{array}{|c|c|c|} \hline \textit{sid} & \textit{cid} & \textit{grade} \\ \hline 1234 & \text{Alg1} & 9 \\ 1235 & \text{Alg1} & 10 \\ 1234 & \text{DB2} & 9 \\ \hline \end{array}$$

Figura 7.2. Rezultatul aplicării selecției pe instanța particular a unei relații

Interogarea $\sigma_{\text{grade} > 8}(\text{Enrolled})$ este echivalentă cu interogarea SQL `SELECT DISTINCT * FROM Enrolled WHERE grade > 8.`

Predicatul de selecție este de forma **Term Op Term** și reprezintă o condiție, unde

- **Term** este un nume de atribut, sau o constantă
- **Op** este un operator logic (ex. $<$, $>$, $=$, $\neq$ etc.).

$(C1 \wedge C2)$, $(C1 \vee C2)$, $(\neg C1)$ sunt condiții formate din operatorii $\wedge$ (și logic), $\vee$ (sau logic) sau $\neg$ (negație), iar $C1$ și $C2$ sunt la rândul lor condiții

Deoarece rezultatul unei interogări este o tot relație, operatorii algebrici pot fi compuși. Astfel putem construi interogări de forma $\pi_{\text{cid}, \text{grade}}(\sigma_{\text{grade} > 8}(\text{Enrolled}))$, echivalentă cu interogarea SQL `SELECT DISTINCT cid, grade FROM Enrolled WHERE grade > 8.`

Reuniune, intersecție, diferență.

$$\begin{aligned} R_1 \cup R_2 &= \{ t \mid t \in R_1 \vee t \in R_2 \} \\ R_1 \cap R_2 &= \{ t \mid t \in R_1 \wedge t \in R_2 \} \\ R_1 - R_2 &= \{ t \mid t \in R_1 \wedge t \notin R_2 \} \end{aligned}$$

Relațiile R_1 și R_2 trebuie să fie *compatibile*:

- același număr de atribute (aceeași *aritate*)
- atributele aflate pe aceeași poziție au domenii *compatibile* și *același nume*

$R_1 \cup R_2$ e echivalent cu `SELECT DISTINCT * FROM R1 UNION SELECT DISTINCT * FROM R2`

$R_1 \cap R_2$ e echivalent cu `SELECT DISTINCT * FROM R1 INTERSECT SELECT DISTINCT * FROM R2`

$R_1 - R_2$ e echivalent cu `SELECT DISTINCT * FROM R1 EXCEPT SELECT DISTINCT * FROM R2`

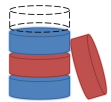
Produs cartezian. Produsul cartezian (notat prin $\times$) reprezintă combinarea a două relații $R_1(a_1, \dots, a_n)$ și $R_2(b_1, \dots, b_m)$ astfel încât:

$$\begin{aligned} R_1 \times R_2 &= \{ t \mid t_1 \in R_1 \wedge t_2 \in R_2 \\ &\quad \wedge t.a_1 = t_1.a_1 \dots \wedge t.a_n = t_1.a_n \\ &\quad \wedge t.b_1 = t_2.b_1 \dots \wedge t.b_m = t_2.b_m \} \end{aligned}$$

Interogarea $R_1 \times R_2$ este echivalentă cu interogarea SQL `SELECT DISTINCT * FROM R1, R2.`

θ -Join. Operatorul θ -join (notat prin $\otimes_c$) reprezintă combinarea a două relații R_1 și R_2 cu respectarea condiției cindicată ca indice.

Interogarea $R_1 \otimes_c R_2$ este echivalentă cu $\sigma_c(R_1 \times R_2)$.



Asemenea, interogarea $\text{Students} \bowtie_{\text{Students.sid}=\text{Enrolled.sid}} \text{Enrolled}$ este echivalentă cu interogările SQL:

```
SELECT DISTINCT * FROM Students, Enrolled WHERE Students.sid = Enrolled.sid
și
SELECT DISTINCT * FROM Students INNER JOIN Enrolled ON Students.sid = Enrolled.sid
```

Equi-Join. Operatorul equi-join (notat prin $\bowtie_{E(c)}$) reprezintă combinarea a două relații pe baza unei condiții compuse doar din egalități ale unor atribute aflate în prima și a doua relație și proiectează doar unul dintre atributele redundante (deoarece valorile celor două atribute sunt egale)

$$R_1 \bowtie_{E(c)} R_2.$$

Courses		$\bowtie_{E(\text{Courses.cid} = \text{Enrolled.cid})}$	Enrolled			=				
cid	cname		sid	cid	grade		cname	sid	cid	grade
Alg1	Algorithms1		1234	Alg1	9		Algorithms1	1234	Alg1	9
DB1	Databases1		1235	Alg1	10		Algorithms1	1235	Alg1	10
DB2	Databases2		1234	DB1	10		Databases1	1234	DB1	10
			1234	DB2	9		Databases2	1234	DB2	9
			1236	DB1	7		Databases1	1236	DB1	7

Figura 7.3. Rezultatul aplicării operatorului equi-join pe instanțele particulare a două relații

Join Natural. Operatorul join natural (notat simplu prin $\bowtie$) reprezintă combinarea două relații pe baza egalității atributelor ce au *același nume* și proiectează doar unul dintre atributele redundante

$$R_1 \bowtie R_2.$$

Courses		$\bowtie$	Enrolled			=				
cid	cname		sid	cid	grade		cname	sid	cid	grade
Alg1	Algorithms1		1234	Alg1	9		Algorithms1	1234	Alg1	9
DB1	Databases1		1235	Alg1	10		Algorithms1	1235	Alg1	10
DB2	Databases2		1234	DB1	10		Databases1	1234	DB1	10
			1234	DB2	9		Databases2	1234	DB2	9
			1236	DB1	7		Databases1	1236	DB1	7

Figura 7.4. Rezultatul aplicării operatorului join natural pe instanțele particulare a două relații

Câtul. Fie R_1 cu 2 atribute, x și y și R_2 având un atribut y . Operatorul cât (notat cu $/$) aplicat relațiilor R_1 și R_2 returnează toate tuplurile x a.î. pentru fiecare dintre tuplurile y din R_2 , există câte un tuplu xy în R_1 .

$$R_1 / R_2 = \{ \langle x \rangle \mid \exists \langle x, y \rangle \in R_1 \quad \forall \langle y \rangle \in R_2 \}$$

Dacă mulțimea valorilor y asociate cu o valoare x din R_1 conține toate valorile y din R_2 , atunci x va fi returnat în rezultat R_1/R_2 . Generalizând, x și y pot reprezenta orice multime de atribute; y este mulțimea atributelor din R_2 , și $x \cup y$ reprezintă atributele lui R_1 .

Ideea: Pentru R_1 / R_2 , vom determina valorile x care nu sunt 'conectate' cu anumite valori y din R_2 .

Valoarea x este *deconectată* dacă atașând la ea o valoare y din R_2 , obținem un tuplu xy ce nu se regăsește în R_1 . Prin urmare *valorile x deconectate* sunt:



$$\pi_x ((\pi_x(R_1) \times R_2) - R_1).$$

Astfel putem rescrie operatorul *cât* folosind operatori algebrici relaționali de bază astfel:

$$R_1 / R_2 = \pi_x(R_1) - (\text{valorile } x \text{ deconectate})$$

sau

$$R_1 / R_2 = \pi_x(R_1) - \pi_x((\pi_x(R_1) \times R_2) - R_1)$$

Redenumirea. Dacă atributele și relațiile au aceleași nume (de exemplu la *join*-ul unei relații cu ea însăși) este necesar să le putem redenumi. Operatorul redenumire se notează prin ρ și are formele alternative:

$$\rho(R' (N_1 \rightarrow N'_1, N_2 \rightarrow N'_2), R)$$

sau

$$\rho_{R' (N'_1, N'_2)}(R),$$

unde noua relație R' are aceeași instanță ca R , iar structura sa conține atributul N'_i în locul atributului N_i .

Interogarea $\rho(\text{Courses2} (cid \rightarrow code, cname \rightarrow description), \text{Courses})$ este echivalentă cu interogarea SQL `SELECT cid as code, cname as description, credits FROM Courses Courses2.`

Aribuirea. Operatorul de atribuire (notat prin $\leftarrow$) oferă un mod simplu de tratare a interogărilor complexe. Atribuirile se fac întotdeauna într-o variabilă temporară:

$$\text{Temp} \leftarrow \pi_x(R_1 \times R_2)$$

Rezultatul expresiei din dreapta $\leftarrow$ este atribuit variabilei din stânga operatorului $\leftarrow$. Variabilele pot fi utilizate apoi în alte expresii, ca de exemplu:

$$\text{result} \leftarrow \text{Temp} - R_3$$

Exercițiu. Determinați numele tuturor studenților cu note la cursuri cu 5 credite

Soluție 1. $\pi_{\text{name}} ((\sigma_{\text{credits}=5}(\text{Courses})) \otimes \text{Enrolled} \otimes \text{Students})$

Soluție 2. $\pi_{\text{name}} (\pi_{\text{sid}}(\pi_{\text{cid}}(\sigma_{\text{credits}=5}(\text{Courses})) \otimes \text{Enrolled}) \otimes \text{Students}).$

Exercițiu. Determinați numele tuturor studenților cu note la toate cursurile

Se utilizează *câtul*; trebuie pregătite structurile relațiilor înainte de a folosi operatorul *cât*:

$$\rho (\text{TempSIDs}, \pi_{\text{sid}, \text{cid}}(\text{Enrolled}) / \pi_{\text{cid}}(\text{Courses})) \\ \pi_{\text{name}}(\text{TempSIDs} \otimes \text{Students})$$

8. Indexarea bazelor de date relaționale

Indecși.

Indecșii sunt fișiere/structuri de date speciale utilizate pentru accelerarea execuției interogărilor.



Un index se crează pe baza unei *chei de căutare*. *Cheia de căutare* e un atribut/set de attribute folosit(e) pentru a căuta înregistrările dintr-o tabelă/fișier. *Cheia de căutare* este diferită de cheia primară / cheia candidat / supercheia

Un index conține o colecție de înregistrări speciale și permite regăsirea eficientă a “*tuturor înregistrărilor tabelii indexate pentru care cheia de căutare are valoarea k*”.

Caracteristicile indecșilor:

Propagarea modificărilor. Adăugarea/ștergerea de înregistrări în tabela indexată implică actualizarea tuturor indecșilor definiți pentru acea tabelă. Orice modificare a valorilor câmpurilor ce aparțin cheii de căutare presupune actualizarea indecșilor bazați pe acea cheie de căutare.

Dimensiunea. Deoarece utilizarea unui index presupune citirea acestuia în memoria internă dimensiune indexului trebuie să fie redusă.

Ce se întâmplă dacă indexul e (totuși) prea mare? O variantă constă în utilizarea unei structuri de indexare parțiale. În același timp se poate indexa fișierul index (stratificat sau pe o structură arborescentă).

Varianțe de structurare a conținutului unui index:

- (1) înregistrările originale ce includ cheia de căutare
- (2) $\langle k, rid \rangle$, rid – referință spre înreg. cu valoarea k a cheii de căutare
- (3) $\langle k, list \rangle$, listă de rid-uri

Alegerea variantei de stocare a intrărilor (înregistrărilor) din index e ortogonală cu tehnica de indexare. Exemple de tehnici de indexare: arbori B+, acces direct.

Indexul mai conține și informații auxiliare de direcționare a căutărilor spre înregistrările căutate.

Varianta (1). Indexul și înregistrările originale sunt stocate împreună. Pentru o colecție de înregistrări se poate crea cel mult un index structurat conform variantei (1) (în caz contrar avem înregistrări duplicate → redundanță → potențiale inconsistențe). Dacă înregistrările sunt voluminoase, numărul de blocuri în care se stochează înregistrările este mare și prin urmare dimensiunea informației auxiliare din index este la rândul său mare.

Variantele (2) și (3): Intrările din index referă înregistrările originale. Intrările din index sunt în general mai mici decât înregistrările (deci sunt variante mai eficiente decât varianta (1), mai ales dacă cheia de căutare este mică). Informația auxiliară din index folosită pentru a direcționa căutarea, este la rândul său mai redusă decât în cazul variantei (1). Varianta (3) este mult mai compactă decât varianta (2), dar implică dimensiune variabilă a spațiului de stocare a intrărilor, chiar dacă cheia de căutare este de dimensiune fixă.

Clasificarea indecșilor.

- A. Indecși *primari* vs *secundari*. Un index *primar* se formează pe baza unei chei de căutare ce include cheia primară. Un index e **unic** atunci când cheia de căutare conține o cheie candidat. Nu vor fi intrări duplicate. În general, indecșii *secundari* conțin duplicate.
- B. Indecși *clustered (grupat)* vs. *un-clustered (negrupat)*. Un index este *clustered* (grupat) dacă ordinea înregistrărilor este aceeași (sau foarte apropiată) cu ordinea



intrărilor din index. Varianta (1) implică grupare; de asemenea în practică, gruparea implică utilizarea primei variante de structurare a indecșilor. O tabelă poate fi indexată grupat (*clustered*) pentru cel mult o cheie de căutare. Costul regăsirii datelor prin intermediul unui index e influențat *decisiv* de grupare!

Pentru a construi indecși grupați:

- Se ordonează înregistrările în cadrul fișierului (*heap file*);
- Se rezervă spațiu în fiecare pagină de memorie, pentru a se absorbi viitoarele inserări dacă spațiul suplimentar e consumat, se vor forma liste înlănțuite de pagini suplimentare.

E necesară o reorganizare periodică pentru a se asigura o performanță ridicată. Întreținerea indecșilor grupați e foarte costisitoare.

C. Indecși *denși* vs. *rari*. Un index este *dens* dacă are cel puțin o intrare pentru fiecare valoare a cheii de căutare (ce se regăsește în înregistrări). Mai multe intrări pot avea aceeași valoare pentru cheia de căutare în cazul utilizării variantei (2) de stocare.

Varianta (1) presupune implicit ca indexul e dens.

Un index e *rar* dacă memorează o intrare pentru fiecare pagină de înregistrări. Toți indecșii rari sunt grupați (*clustered*)! Indecșii rari sunt mai compacți.

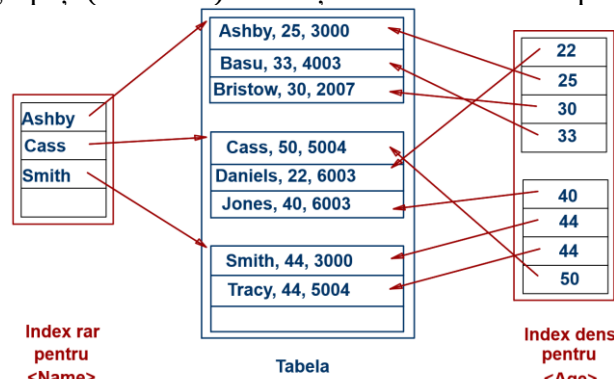


Figura 8.1. Exemplu de index rar și dens definiți pe aceeași instanță

D. Indecși cu chei de căutare *simple* vs. *compuse*. Chei de căutare compuse (formate din mai mult de un atribut) sunt utile atunci când căutarea se realizează după o combinație de câmpuri. De exemplu, interogări cu egalitate (valoarea fiecărui câmp e egală cu o valoare constantă - *age=20* și *sal=250* sau interogări cu interval (valoarea unui câmp nu e o constantă - *age=20* și *sal > 30*).

Intrările din index sunt sortate după cheia de căutare (în ordine lexicografică sau spațială) pentru a putea fi utile interogărilor cu interval.

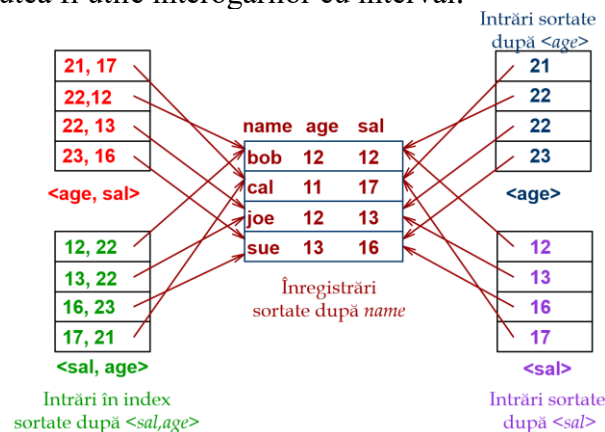
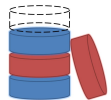


Figura 8.1. Exemplu de index rar și dens definiți pe aceeași instanță



E. Indecși multinivel (indexare multiplă)

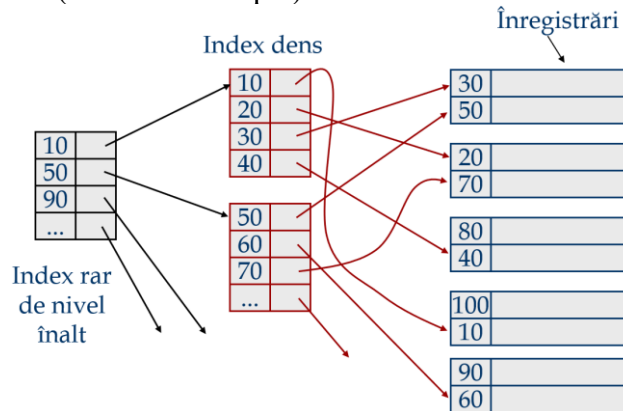


Figura 8.2. Exemplu de indexare multinivel

Alegerea și implementarea indecșilor.

Înainte de a implementa un index este necesară analiza interogărilor și operațiilor de actualizare a înregistrărilor. Astfel, pentru fiecare interogare utilizată va trebui să răspundem la următoarele întrebări:

- Care sunt relațiile/tabelele accesate?
- Care sunt atributele/câmpurile returnate?
- Care dintre atribute sunt implicate în condiții de selecție/join? Cât de selective sunt aceste condiții?

De asemenea pentru fiecare operație de actualizare trebuie determinat:

- Care dintre atribute sunt implicate în condiții de selecție/join? Cât de selective sunt aceste condiții?
- Ce tip de actualizare este (INSERT/DELETE/UPDATE), și care sunt atributele afectate?

Vom deduce astfel care sunt comenzile care se execută cel mai des și care vor fi optimizate cu prioritate.

Se analizează cele mai importante interogări. Pentru fiecare se consideră cel mai optim plan de execuție folosind indecșii existenți. Vom crea un nou index doar dacă acesta va genera un plan mai bun (evident, acest lucru implică să înțelegem cum evaluează un SGBD interogările și cum sunt create planurile de execuție).

Înainte de a crea un nou index, trebuie să evaluăm impactul actualizării acestuia! Indecșii accelerează execuția interogărilor, dar încetinesc actualizările. De asemenea, necesită spațiu suplimentar de stocare.

Atributele din clauza WHERE sunt chei de căutare “candidat”. Egalitățile sugerează o structură cu acces direct iar condițiile pe intervale sugerează utilizarea unei structuri arborescente. Gruparea e foarte utilă în aceste situații; dar și în cazul egalităților atunci când numărul de duplicate este mare.

Atunci când clauza WHERE are mai multe condiții se pot lua în considerare chei de căutare compuse, caz în care ordinea atributelor e importantă pentru a evalua condițiile cu intervale.

Pentru interogările importante acești indecși pot determina strategii de execuție *index-only*.

Vor fi aleși indecșii de care beneficiază cele mai multe interogări posibile. Deoarece se poate defini un singur index grupat pe o tabelă, acesta va fi ales astfel încât să beneficieze de acest index o interogare foarte importantă



9. Organizarea arborescentă a indecșilor

Fișiere *heap* (nesortate) sau mod de memorare sortat al înregistrărilor sunt modalități utile de stocare a tabelor statice. Atunci când modificările operate pe tabele sunt frecvente, se recomandă utilizarea unor structuri mai complexe de reprezentare.

În practică tabelele sunt stocate ca fișiere *heap* iar indecșii sunt structurați astfel încât să permită optimizarea atât a regăsirii cât și a modificărilor de date.

Arbori binari

Arborii binari sunt eficienți pentru inserare /ștergere înregistrări și permit utilizarea unor algoritmi rapizi de căutare binară.

Structura memoriei pentru arbore binar (figura 9.1):

- colecție de noduri; referință rădăcină
- listă de noduri libere (înlănțuite prin $Pointer_{Left}$)

K	Data	Pointer _{Left}	Pointer _{Right}
---	------	-------------------------	--------------------------

Figura 9.1. Structura memoriei pentru un nod de arbore binar (K reprezintă atributele ce compun cheia, $Data$ reprezintă toate celelalte atribute ale tablei, iar pointerii vor referi rădăcinile subarborilor)

Figura 9.2 prezintă un arbore binar particular însoțit de reprezentarea sa în memoria calculatorului.

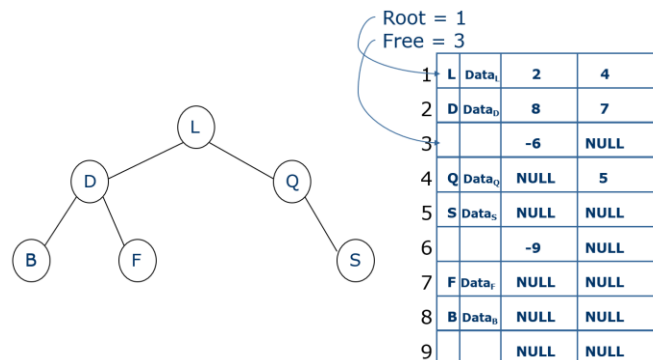


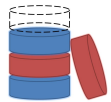
Figura 9.2. Reprezentarea în memorie a unui arbore binar ($Root$ e referință la rădăcina arborelui iar $Free$ este o referință capul listei de noduri libere)

La inserarea unei înregistrări se vor urma pașii:

- detectare poziție înregistrare;
- stocare înregistrare nouă într-un nod liber;
- legare nod la părinte.

Pentru eliminarea unei înregistrări se vor realiza:

- căutare înregistrare;
- odată înregistrarea găsită se va acționa în funcție de unul dintre următoarele cazuri:
 - o înregistrarea e frunză (fără copii): pointer corespunzător din părinte = NULL;
 - o înregistrarea are un subarbore: asignează referință subarbore la părinte;



- înregistrarea are doi subarbori: se înlocuiește cu cel mai apropiat vecin și se reiau pașii pentru eliminarea acestui nod;
- la final nodul eliminat se va adăuga în capul listei de noduri libere (*Free* va referi acest nod).

Pașii descriși mai sus pot dezechilibra arborele binar în timp, implicând timpi mai mari de căutare (timpul de căutare depinde de ordinea în care înregistrările au fost inserate în tabel).

Arbori optimali

Arborii optimali sunt arbori binari în care frunzele sunt poziționate pe cel mult 2 nivele. Întreținerea acestor arbori este dificilă și implică un consum de timp ridicat pentru adăugare, ștergere sau actualizare a arborelui.

Arbori binari echilibrați

Arborii echilibrați sunt arbori binare în care pentru fiecare nod diferența dintre înălțimile subarborilor săi este 0, 1 sau -1. Prin *înălțimea unui arbore* înțelegem dimensiunea celui mai lung drum de la rădăcina arborelui la frunzele acestuia.

Arborii echilibrați implică un număr redus de operații de întreținere având 6 cazuri distincte de “dezechilibrare” a acestora după inserarea unei noi valori (figurile 9.3 și 9.4)

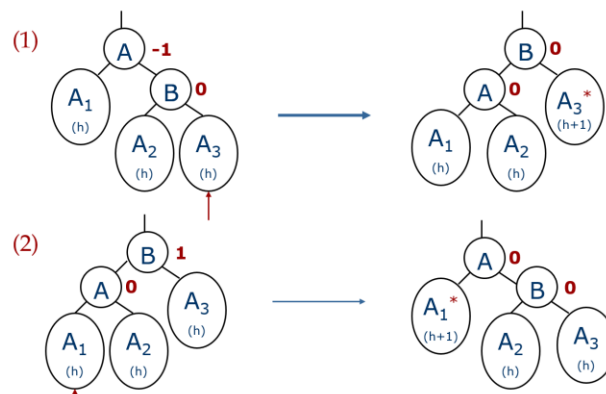
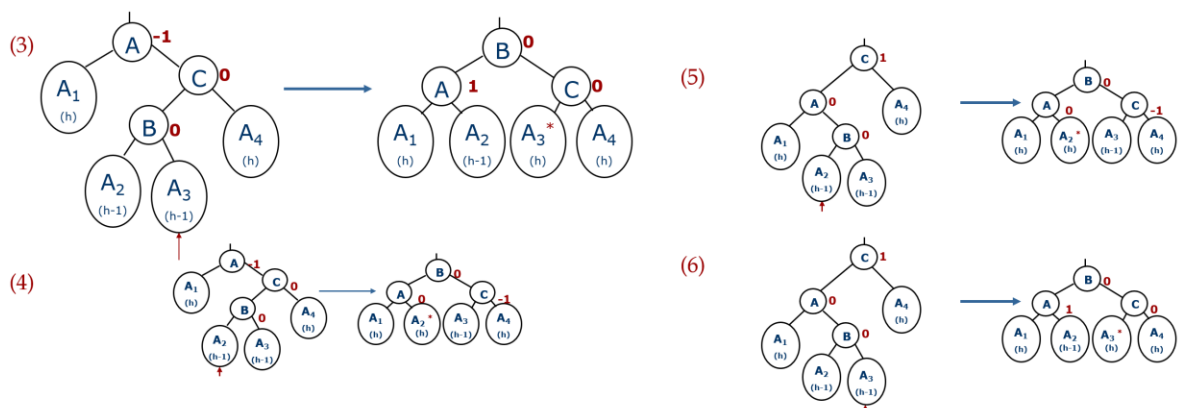


Figura 9.3. Două cazuri similare de dezechilibrare a unui arbore echilibrat a cărui re-echilibrare implică 2 atribuiri



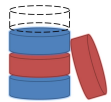


Figura 9.4. patru cazuri similare de dezechilibrare a unui arbore echilibrat
a cărui re-echilibrare implică 4 atribuiri

Metoda de acces secvențial indexată

ISAM (*Indexed Sequential Access Method*)

Să presupunem că vrem să executăm o interogare ce returnează răspunsul la următoarea întrebare: "Returnează toți studenții cu **grade** > 8.0"

Dacă datele sunt stocate într-un fișier ordonat, prin căutare binară se detectează primul student, și apoi sunt citite toate înregistrările următoare însă costul căutării/întreținerii poate fi ridicat.

Soluția ar putea consta în crearea unui fișier index ca cel din figura 9.5, în care un pointer P_i să refere o pagină de memorie ce conține toate înregistrările a căror valoare pentru cheia de căutare se află între K_i și K_{i+1} .

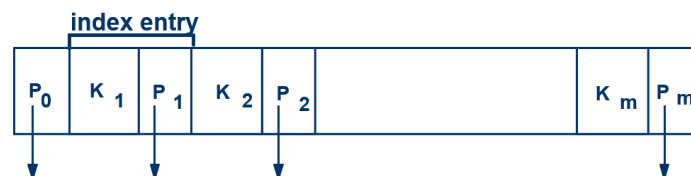


Figura 9.5. Metoda de acces secvențial indexată –
reprezentare în memorie a intrărilor din index

Fișierul index poate fi mare, însă aceeași idee se poate aplica repetitiv (figura 9.6)

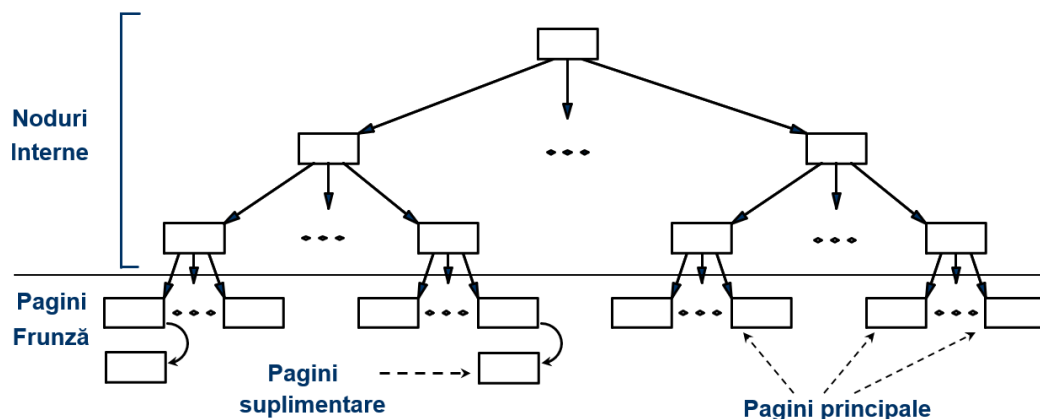


Figura 9.6. Aplicarea repetitivă a indexării, înregistrările din tabele fiind memorate în paginile frunză sau (dacă e cazul) în pagini suplimentare

Operații pe fișiere ISAM:

- *Creare fișier*: paginile (de date) frunză sunt alocate secvențial, sortate după cheia de căutare; apoi se memorează paginile index, apoi spațiul pentru paginile suplimentare.
- *Intrări index* : <val cheie căutare, id pagină>; vor 'direcționa' căutarea către *paginile de date* (frunze).
- *Căutare*: Comparări ale cheii pornind de la rădăcina spre frunze. Cost $\log_F N$; F = număr intrări/pg index, N = numărul paginilor frunză
- *Inserare*: Căutare pagină frunză și adăugare înregistrare. Dacă e cazul se alocă p pagină suplimentară nouă.



- **Ștergere:** Găsire și eliminare înregistrare din frunză; dacă e cazul, se dealocă o pagina suplimentară (*overflow*)

Prin procedeul descris anterior se obține o structură arborescentă statică unde operațiile de *inserarea/ștergere* afectează doar frunzele.

Exemplu:

Vom lua în considerare un exemplu în care fiecare nod poate stoca 2 înregistrări (și 3 pointeri). Fișierul index are configurația din figura 9.7.

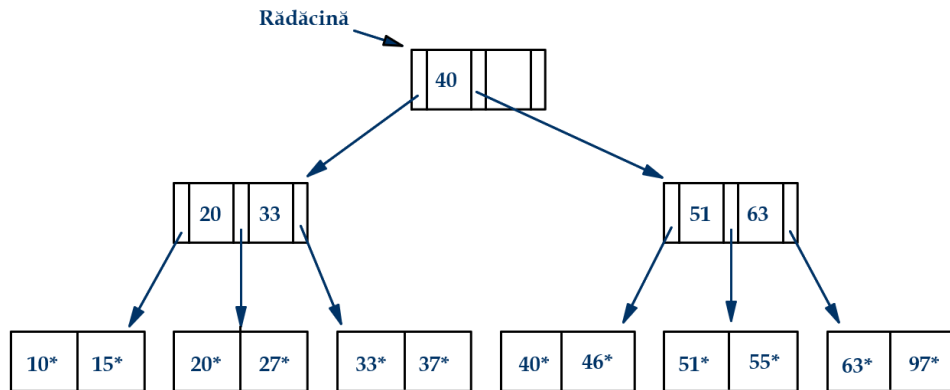


Figura 9.7. Structură de fișier ISAM, unde nodurile frunză conțin înregistrări complete (am condicivat acest lucru folosind caracteru *) iar nodurile intermediare conțin valori ale chvilor de căutare

După inserarea înregistrărilor ce au valorile 23, 48, 41, respectiv 42 pentru cheia de înregistrare, fișierul index va avea configurația din figura 9.8.

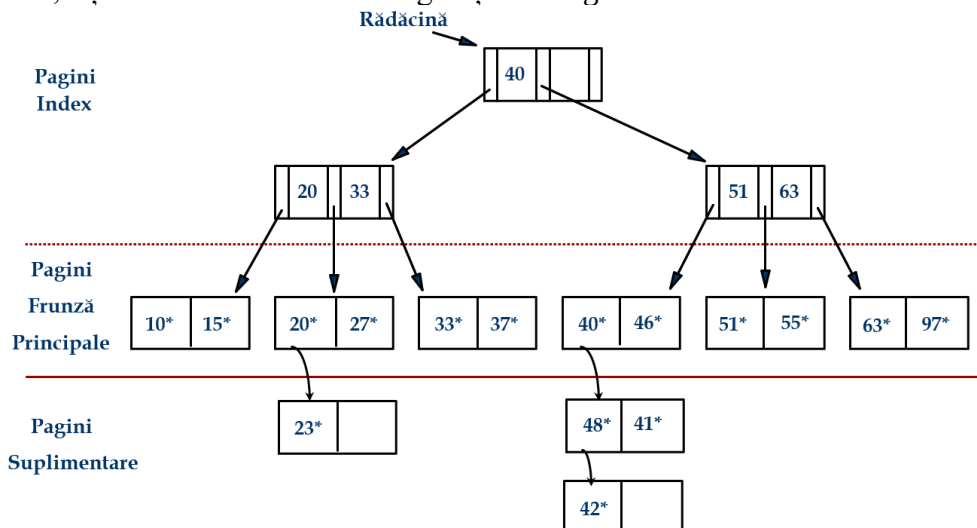


Figura 9.8. Structura fișierului ISAM după adăugarea înregistrărilor 23*, 48*, 41*, 42* ...

De asemenea, după eliminarea înregistrărilor ce au valorile 42, 51 și 97 pentru cheia de înregistrare, fișierul index va avea configurația din figura 9.9.

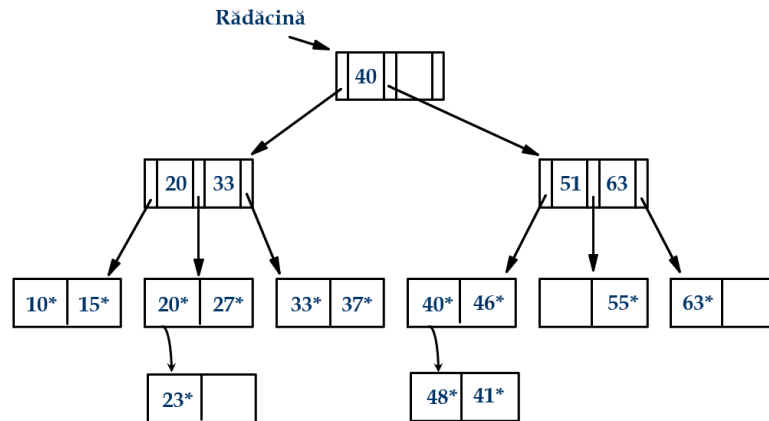
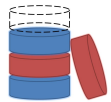


Figura 9.9. Structura fișierului ISAM după ștergerea înregistrărilor 42*, 51*, 97*

Se observă că valoarea 51 apare în unul dintre nodurile interne ale structurii indexului, dar nu mai există în frunze deoarece înregistrarea a fost eliminată.

Structurile ISAM au unele avantaje importante:

- operațiile de inserare & ștergere sunt rapide și acestea nu conduc la dezechilibrarea structurii arborescente;
- nivelul de concurență al accesului este crescut deoarece nodurile arborelui nu sunt blocate niciodată (acestea nu se vor modifica).

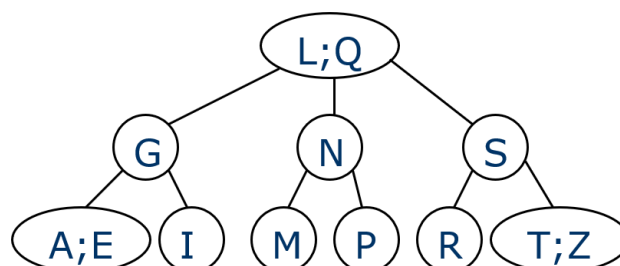
Totuși aceste structuri de indexare sunt potrivite doar pentru tabele al căror conținut se modifică rar, deoarece pot apărea dezechilibrări în frunze (unele frunze vor avea mai multe pagini suplimentare asociate, iar altele mai puține). În plus, înregistrările din paginile suplimentare nu sunt de obicei sortate (dar ar putea fi), căutarea în aceste pagini fiind secvențială.

Atunci când se constată timpi mai mari de regăsire și acces al datelor, structurile ISAM sunt șterse și refăcute pe baza noului conținut al tabelelor indexate.

Arbori-B

Arborii-B reprezintă o soluție de combinare a avantajelor date de structurile ISAM și a arborilor echilibrați. Arborii-B reprezintă cea mai populară metodă de organizare a indecșilor în baze de date. “B” vine de la “*balanced*” sau “*broad*”

Un arbore-B este un arbore ordonat. Fiecare nod conține chei (ordonate în interiorul nodului) și o listă de pointeri la sub-arbori. O proprietate importantă a arborilor-B este aceea că drumurile de la rădăcină la frunze au aceeași lungime (cu alte cuvinte toate frunzele se află pe același nivel – figura 9.10)



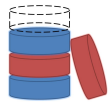


Figura 9.10. Exemplu de arbore-B de ordin 3

Într-un arbore-B de ordin m :

- dacă nu e frunză, rădăcina va avea mereu cel puțin 2 subarbori
- fiecare nod intern are cel puțin $\lceil m/2 \rceil$ sub-arbori (mai puțin rădăcina)
- fiecare nod intern are cel mult m sub-arbori
- toate frunzele sunt la același nivel

Un nod cu p sub-arbori conține $p-1$ chei ordonate ($K_1, K_2, \dots, K_p$) – figura 9.11

- subarborele T_1 conține valori $< K_1$
- subarborele T_i conține valori între K_{i-1} și K_i
- subarborele T_p conține valori $> K_p$

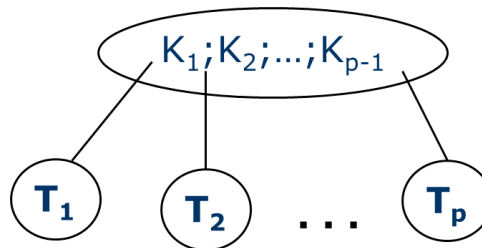


Figura 9.11. Structura unui nod de arbore-B

Organizarea în memorie a Arborilor-B

A. Ca arbore binar

$Pointer_{Left}$

referă prima cheie a sub-arborelui stâng din Arborele-B

$Pointer_{Right/H}$

referă vecinul din dreapta în nodul Arborelui-B

referă prima cheie a sub-arborelui drept din Arborele-B (dacă e ultima valoare a cheii în nodul Arborelui B)

Flag adițional / utilizare semn pentru *pointer* dreapta

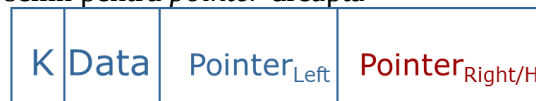


Figura 9.12.



Figura 9.13.

B. Alocare memorie pentru a stoca $m-1$ chei per nod

N – număr de chei stocate

K_i – valoarea cheii, AD_i – adresa înregistrării

$Point_i$ – referă un sub-arbore

Utilizarea spațiului de memoriei

pentru *pointeri* la stocarea cheilor

în frunze



$m/2 \leq N \leq m-1$ – pentru noduri interne

$m/2 \leq N \leq t$ – pentru noduri terminale

flag adițional/ semn pentru N

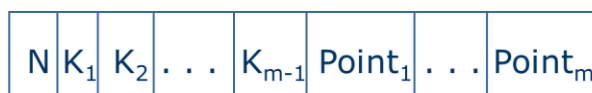


Figura 9.14. Structura unui nod de arbore-B

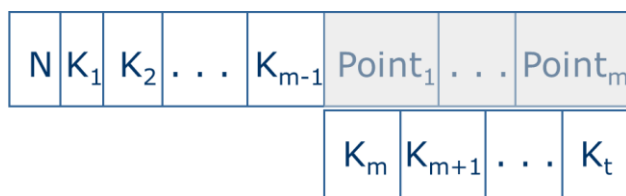


Figura 9.15. Structura unui nod de arbore-B

Căutarea într-un Arbore-B

Exemplu: căutare '15'

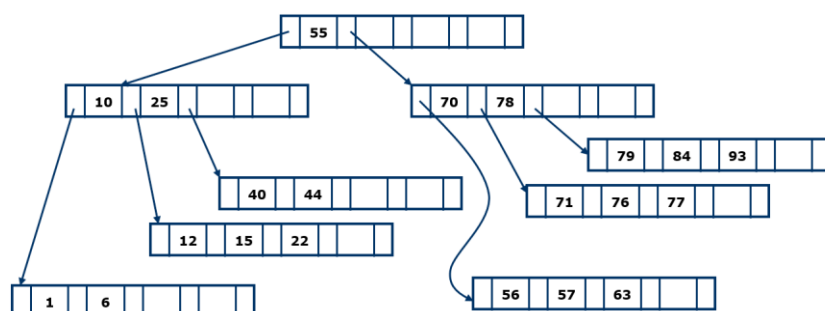


Figura 9.16. Structura unui nod de arbore-B

Inserarea înregistrărilor în Arbore-B

Pași de inserare:

localizare nod unde cheia trebuie să fie inserată

se inserează noua cheie

se aplică o procedură de *echilibrare* în cazul în care este depășit numărul maxim de chei ce pot fi stocate în nod

Algoritmul procedurii de inserare

1. Localizare nod pentru inserare

2. Inserare cheie

3. Dacă nodul e plin (dimensiunea e depășită):

A) Se crează un nou nod în care se mută cheile mai mari decât valoarea cheii mediane

B) Se inserează cheia mediană în nodul părinte

C) *Pointerul* din dreapta cheii va referi noul nod, iar cel din stanga referă vechiul nod ce conține valorile mai mici

4. Dacă și nodul părinte e plin:

A) Dacă nodul părinte e rădăcină atunci se crează o rădăcină nouă

B) Se repetă pasul 3 pentru nodul părinte



Ștergerea înregistrărilor în Arbore-B

Pași de ștergere:

- se indentifică nodul ce conține valoarea ce trebuie ștearsă
- dacă e un nod intern, *se transferă* o valoare din frunze
- în caz de *subdimensionare*, se realizează o *redistribuire* sau o *concatenare*

Algoritm de ștergere

1. Se caută valoarea ce trebuie ștearsă.

Dacă se află într-un nod intern se înlocuiește cu valoarea *vecină mai mare*
(adică cu cea mai din stânga valoare
a celei mai din stânga frunze
a subarborelui drept)

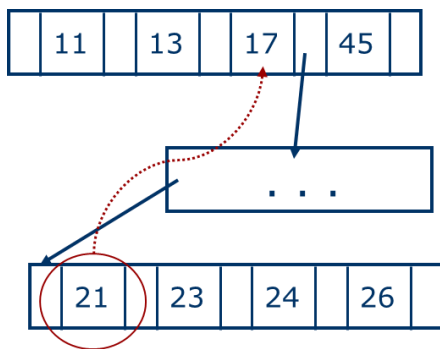


Figura 9.17. Structura unui nod de arbore-B

2. Se repetă acest pas până se ajunge în cazurile A) sau B)

A) Dacă nodul ce conține valoarea de ștears este rădăcina sau numărul valorilor rămase în nod este $\geq \lceil m/2 \rceil$:
se elimină valoarea
se *re-aranjează valorile și pointerii* din nod
se termină algoritmul

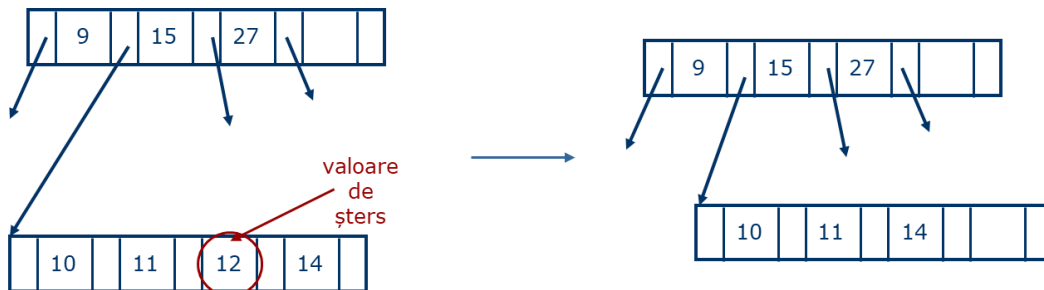


Figura 9.18. Structura unui nod de arbore-B

B) Dacă numărul valorilor rămase în nod este $< \lceil m/2 \rceil$ unul dintre nodurile vecine conține $> \lceil m/2 \rceil$ valori $\rightarrow$ *redistribuire*
se ordonează valorile din ambele noduri + valoarea separator din părinte
se alege valoarea mediană și se adaugă la nodul părinte, iar celelalte valori se inserează în nodul stâng, respectiv drept
algoritmul se termină

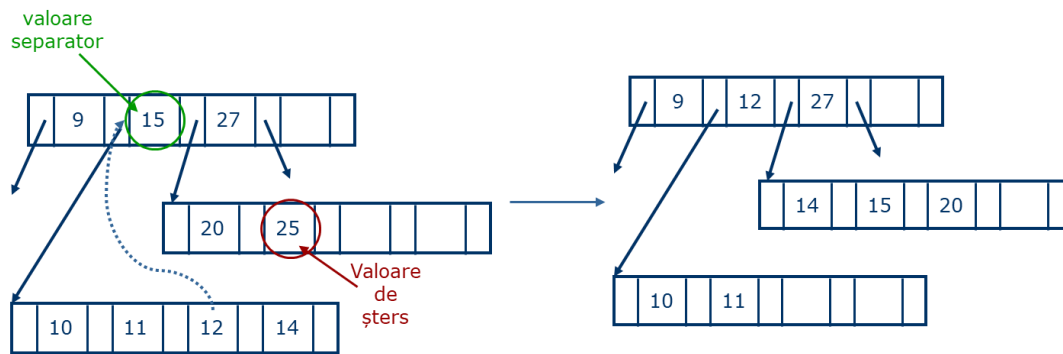


Figura 9.19. Structura unui nod de arbore-B

C) Dacă suma valorilor din nodul din care s-a eliminat valoarea și a valorilor unuia din vecini este $< m \rightarrow$ concatenare

se inserează valorile ambelor noduri + valoarea separator din nodul părinte într-un singur nod

se repetă pasul 2. pentru nodul părinte (din care s-a eliminat valoarea separator)

dacă nodul părinte este rădăcina și nu mai conține valori $\rightarrow$ nodul curent devine rădăcina

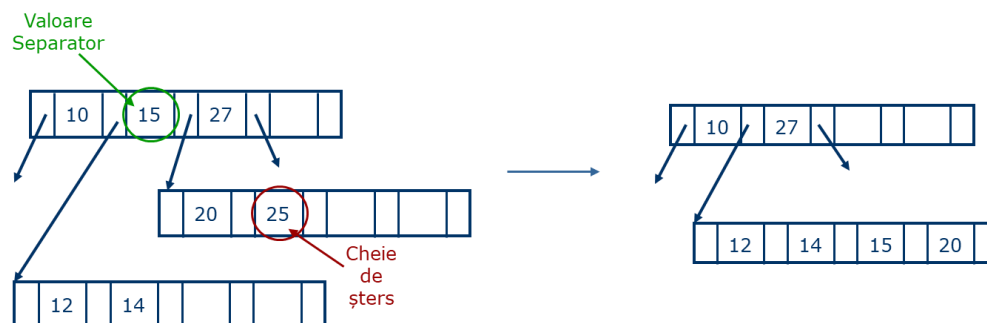


Figura 9.20. Structura unui nod de arbore-B

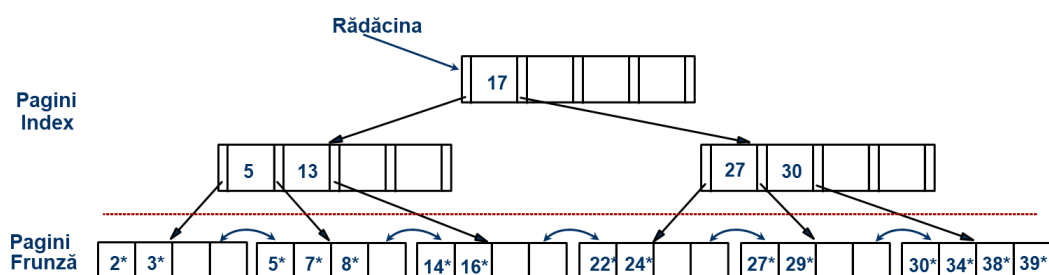
Arbori B+

Combinație între Arbori-B și ISAM:

Căutarea pornește de la rădăcină, și va fi direcționată prin comparații către o frunză

Într-un Arbore B+ toți pointerii către înregistrări din tabele se află doar la nivelul nodurilor frunză

Un arbore B+ poate avea mai puține nivele (sau o capacitate mai mare pentru stocarea cheilor de căutare) decât arborele B corespondent



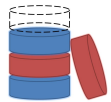


Figura 9.21. Structura unui nod de arbore-B

Structura nodurilor

Noduri interne

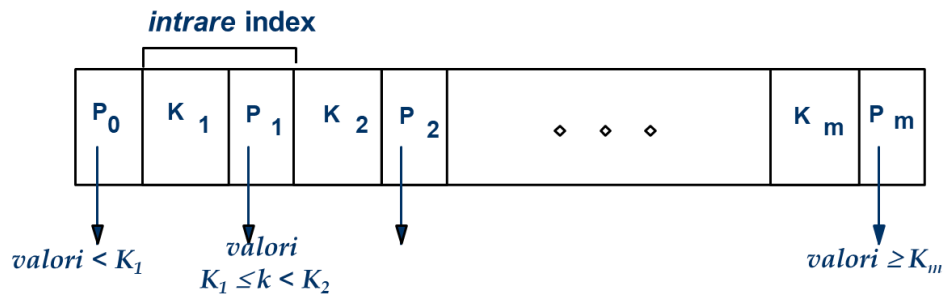


Figura 9.22. Structura unui nod de arbore-B

Noduri frunză

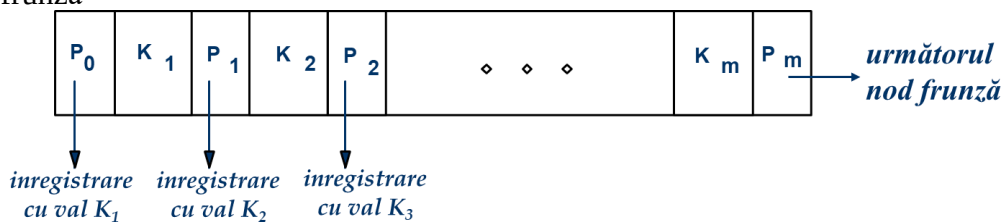


Figura 9.23. Structura unui nod de arbore-B

Arborii B+ în practică

Ordin tipic: 200. Factor de acoperire: 67%.

În medie, (nr de valori)/(nr pagini de index) = 133

Capacități tipice:

Înălțime 4: $133^4 = 312,900,700$ înregistrări

Înălțime 3: $133^3 = 2,352,637$ înregistrări

În general poate fi memorat în *buffer*-ul din memoria internă:

Nivel 1 = 1 pagină = 8 Kbytes

Nivel 2 = 133 pagini = 1 Mbyte

Nivel 3 = 17,689 pagini = 133 MBytes

Avantaje & Dezavantaje ale Arborilor-B+

Indecșii rămân echilibrați → timp de căutare uniform

Rareori sunt mai mult de 3 - 5 nivele → primele nivele sunt păstrate în RAM astfel încât o căutare va necesita doar 2 sau 3 I/O (citiri/scrieri).

În general nodurile au o ocupare de 67% (deci se folosește cu 50% mai mult spațiu decât este necesar)

Datorită versatilității sale este cel mai utilizat mod de a structura indecșii în SGBD-uri relaționale. E una dintre cele mai optimizate componente ale unui SGBD.

Arborii B+ pot fi utilizați pentru indecși *clustered*, rari (dacă tabela e sortată) sau pentru indecși *un-clustered*, denși (în caz contrar).

Bulk Loading

În cazul unei colecții mari de înregistrări crearea unui arbore B+ prin inserarea repetată a fiecărei înregistrări este lentă.

Procedura de Bulk Loading este mult mai eficientă.



Initializare: Se sortează toate înregistrările tabelului și se inserează un pointer către prima pagină într-o nouă pagină a arborelui (rădăcina).

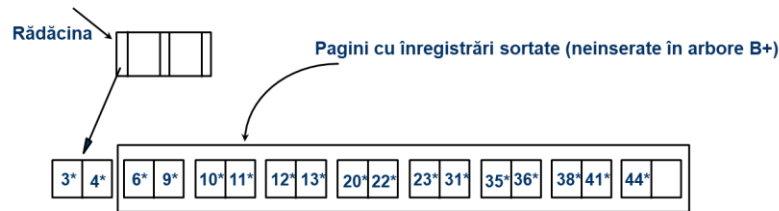


Figura 9.24. Structura unui nod de arbore-B

Intrările din index sunt întotdeauna inserate în cea mai din dreapta pagină deasupra ultimului nivel. Când pagina devine plină se împarte în două. Mult mai rapid decât prin inserări repetate!

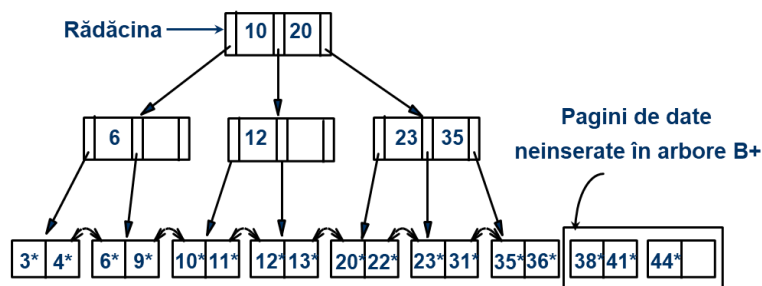


Figura 9.25. Structura unui nod de arbore-B

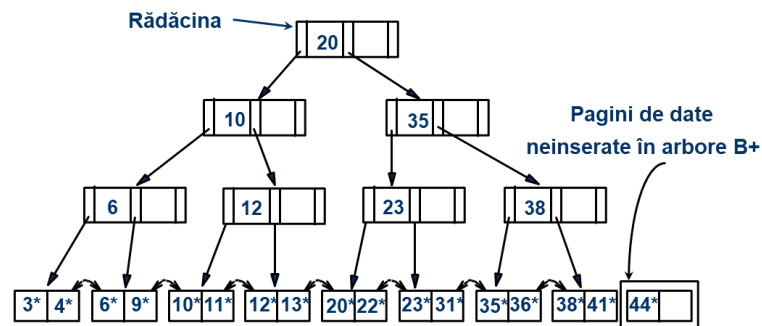


Figura 9.26. Structura unui nod de arbore-B

Sumar *Bulk Loading*

Opțiune 1: inserări multiple.

Lent.

Frunzele nu sunt stocate secvențial.

Opțiune 2: *Bulk Loading*

Indexul poate fi utilizat concurrent.

Mai puține citiri/scrieri (I/Os) în timpul construcției

Frunzele sunt stocate secvențial (și înlănțuite).

Se poate controla “gradul de umplere” a unei pagini

Arbore B+ Prefix (compresie de cheie)



Conduce la creșterea numărului de valori stocate într-un nod

Valorile din index sunt folosite doar pentru a direcționa "traficul" comparațiilor, și de aceea pot fi comprimate

Ex. Dacă avem intrări adiacente în index cu următoarele valori pentru cheia de căutare *Dan Yogurt*, *David Smith* și *Demy Moore*, putem abrevia *David Smith* cu *Dav.* (și de asemenea celelelalte chei ...)

Inserările/ștergerile se pot modifica corespunzător.

10. Organizarea directă a indecșilor

Organizarea directă este o metodă de determinare a adresei unei înregistrări pe baza valorii unuia sau mai multor câmpuri (reprezentând, în general, cheia)

Caz ideal: utilizarea unei funcții ce calculează adresa înregistrării (*funcție de dispersie*):

$h: \{K_1, K_2, \dots, K_n\} \rightarrow A,$

$h(K_i) = \text{adresa celei de-a } i\text{-lea înregistrări}$

(K_i e valoarea cheii înregistrării i , A este mulțimea de adrese de memorie)

Organizarea directă este o metodă de determinare a adresei unei înregistrări pe baza valorii unuia sau mai multor câmpuri (reprezentând, în general, cheia)

Caz ideal: utilizarea unei funcții ce calculează adresa înregistrării (*funcție de dispersie*):

$h: \{K_1, K_2, \dots, K_n\} \rightarrow A,$

$h(K_i) = \text{adresa celei de-a } i\text{-lea înregistrări}$

(K_i e valoarea cheii înregistrării i , A este mulțimea de adrese de memorie)

Organizarea directă este o metodă de determinare a adresei unei înregistrări pe baza valorii unuia sau mai multor câmpuri (reprezentând, în general, cheia)

Caz ideal: utilizarea unei funcții ce calculează adresa înregistrării (*funcție de dispersie*):

$h: \{K_1, K_2, \dots, K_n\} \rightarrow A,$

$h(K_i) = \text{adresa celei de-a } i\text{-lea înregistrări}$

(K_i e valoarea cheii înregistrării i , A este mulțimea de adrese de memorie)

În practică: e greu de definit astfel de funcții:

trebuie cunoscute de la început toate valorile posibile ale cheilor

pentru tabele mari este aproape imposibil de menținut bijectivitatea

Soluție → sunt permise coliziunile:

$h(K_i) = h(K_j), \text{ pt. } i \neq j \quad (h - \text{'funcție de dispersie'})$

Înregistrările cu valorile pentru cheie K_i și K_j se numesc *sinonime*

Toate înregistrările sinonime sunt stocate într-o partiție care începe la adresa returnată de h .

O partiție este de obicei un bloc de memorie.

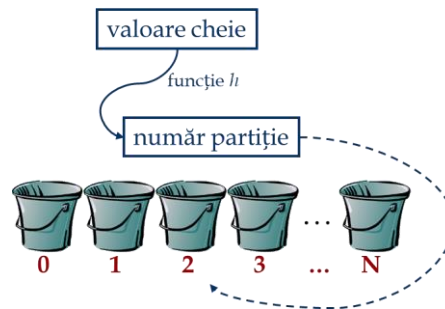


Figura 10.1.

Provocări ale organizării directe

Distribuirea – odată ales algoritmul de dispersie se pierde controlul asupra distribuirii înregistrărilor pe partiții.

Gruparea – majoritatea înregistrărilor sunt plasate în aceeași partiție iar restul partițiilor conțin doar puține înregistrări.

Depășirea – este posibilă depășirea spațiului de memorie alocat inițial unei partiții.

Definirea funcțiilor de dispersie

Cerințele pentru o funcție de dispersie ‘bună’:

- evaluare rapidă
- minimizează numărul de coliziuni

(dispersie uniformă a înregistrărilor pe partiții)

Exemplu:

Fie 41 partiții goale unde adăugăm înregistrări:

- probabilitatea plasării înregistrării 1 într-o partiție goală e 41/41
- pentru ca și a doua înregistrare să fie plasată într-o partiție goală avem probabilitatea 40/41,
- pentru a treia înregistrare - 39/41, etc.

Probabilitatea plasării a 8 înregistrări în 8 partiții goale este :

$$(41/41)(40/41)(39/41)\dots(34/41) = 0.482$$

⇒ **sub 50%!!!**

Abordări:**Diviziune**

Funcții de forma $h(k) = k \bmod N$

Garantează plasarea rezultatului $h(k)$ în intervalul $[0 \dots N-1]$

E ușor de calculat ⇒ cea mai populară abordare

Dacă alegem $N = 2^d$, pentru un d număr natural oarecare, doar ultimii d biți ai lui k vor fi luați în considerare

S-a arătat că numerele prime sunt cele mai potrivite ca valori ale lui N

Metoda pătratului (mid-square)

Se ridică numărul la pătrat apoi se extrag l cifre din mijlocul rezultatului.

$l = \text{lung}(k^2) - c_1 - c_2$, unde c_1 și c_2 sunt reprezentă numărul celor mai puțin semnificative, respectiv cele mai semnificative cifre eliminate din pătratul lui l

l și c_1 sunt fixate de la început

**Împachetare (folding)**

Fiecare cheie k este partiționată în bucăți de lungimi egale (cu excepția ultimei) $k_1, k_2, k_3, \dots, k_n$

Funcția de dispersie e dată de formula

$$h(k) = (k_1 + k_2 + k_3 + \dots + k_n) \bmod 10^l$$

unde 10^l e numărul maxim de partiții

Multiplicare

Se extrage partea fracționară a numărului $Z * k$ (pentru un Z specific) și se înmulțește cu N (N fiind numărul partițiilor):

$$h(k) = \lfloor N * (Z * k - \lfloor Z * k \rfloor) \rfloor = \lfloor N * \{Z * k\} \rfloor$$

Cele mai bune rezultate se obțin pentru

$$Z = (5 - 1)/2 = 0.61803... \text{ sau } Z = (3 - \sqrt{5})/2 = 0.38196...$$

Pentru $Z = Z'/2^w$ și $N = 2^d$ (w : număr de biți într-un cuvânt)

$$h(k) = \text{msb}_d(Z' * k)$$

unde $\text{msb}_d(x)$ reprezintă cei mai semnificativi d biți ai lui x

(exemplu: 42 a reprez. binară 101010, $\text{msb}_3(42) = 5$ – binar 101)

Teoremă. Având un număr irațional x și plasând $\{x\}, \{2x\}, \dots, \{nx\}$ pe segmentul $[0,1]$ se obțin $n+1$ segmente ce au cel mult 3 dimensiuni diferite.

De asemenea, următoarea valoare, $\{(n+1)x\}$, va fi plasată pe unul dintre segmentele cu dimensiunea cea mai mare.

În general funcțiile de dispersie se aplică pe **reprezentarea binară** a cheii de căutare (de exemplu, pentru o cheie de căutare de tip text, se pot însuma reprezentările binare ale tuturor caracterelor acesteia iar rezultatul este transmis ca parametru funcției de dispersie).

Exemplu:

Valoare cheie 'Toyota'

Concatenând reprezentarea numerică a caracterelor de pe primele două poziții obținem:

Toyota $\Rightarrow$ **2015**

Funcții de dispersie posibile

Diviziune: mod by 97 $\rightarrow$ **2015 mod 97 = 75**

Metoda pătratului: **2015² = 4060225**

Împachetare: **2015 $\rightarrow$ 20+15 = 35**

Multiplicare: $\lfloor 99 * \{2015 * 0.61803\} \rfloor = 32$

De ce nu se utilizează **2015** ca valoare de dispersie?

4 cifre $\rightarrow$ 10000 valori posibile! $\rightarrow$ o bună parte a partițiilor vor fi goale
pentru cazul nostru 100 partiții sunt suficiente

Rezolvarea coliziunilor

Noile înregistrări se memorează într-o zonă suplimentară

Se aplică o a doua funcție de dispersie pentru a obține o adresă "second choice".

Se memorează doar referințe (pointeri). La adresa returnată de funcția de dispersie avem:

Toți pointerii înregistrărilor sinonime – **partiție** de adrese.

Pointer către prima înregistrare. Aceasta va conține la rândul său un pointer către a doua înregistrare sinonimă, ș.a.m.d. – **listă înlănțuită**.



Dispersie statică vs. dispersie dinamică

Dispersie statică

Numărul paginilor/sloturilor *principale* e fix

Paginile/sloturile sunt alocate secvențial, și nu sunt de-alocate; se pot utiliza pagini/sloturi *suplimentare* dacă este nevoie

$h(k) = k \bmod M$ = partiția căreia îi aparține intrarea cu cheia k . (M = număr de partiții)

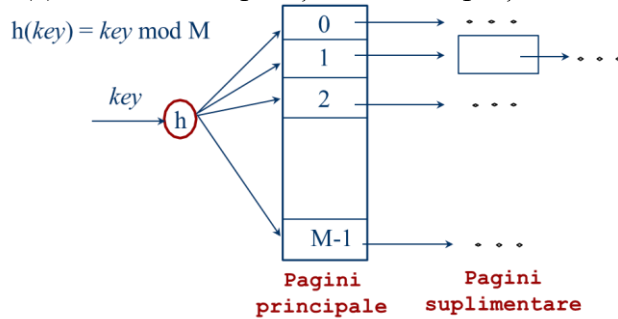


Figura 10.2.

Dispersie statică: Liste independente

Toate sinonimele sunt stocate într-o listă înlănțuită specifică

Fișierul conține o listă de M intrări reprezentând capul unei liste de sinonime (partiții)

Ordinea sinonimelor în listă poate fi:

ordinea de inserare

ordine descrescătoare a frecvenței de căutare

ordinea crescătoare a cheii de căutare (căutările fără succes se vor termina mai rapid)

k	$h(k) = k \bmod 7$
11	4
2	2
44	2
4	4
15	1

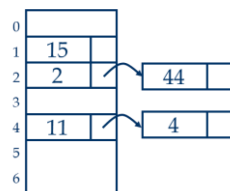


Figura 10.3.

Dispersie statică: Liste întreșesute

Nu se utilizează pagini/sloturi suplimentare

La inserarea unei noi intrări cu valoare cheii K :

Dacă slotul de la adresa $h(K)$ e gol: se memorează înregistrarea

Dacă slotul de la adresa $h(K)$ e ocupat:

se ocupă primul slot liber (căutând de la baza fișierului)

slotul ocupat e inserat la finalul listei ce conține slotul referit de $h(K)$

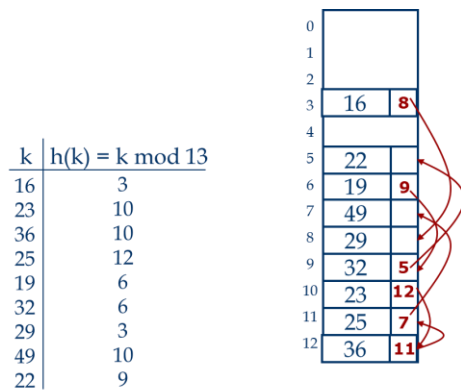


Figura 10.4.

Ștergerea unei înregistrări cu valoarea cheii K :

Dacă slotul de la adresa $h(K)$ e gol: mesaj de eroare

Dacă slotul $h(K)$ e ocupat:

- parcurgem lista pentru a căuta înregistrarea. Dacă aceasta nu e găsită: mesaj de eroare

1. Dacă înregistrarea e găsită: se elimină
2. Se caută, în continuarea listei, o înregistrare r cu $h(K_r) = h(K)$

dacă e găsită

se mută înregistrarea în slotul curent.

se repetă pasul 2 pentru noul slot gol

3. se copiază pointerul slotului gol în precedentul element din listă (dacă există)

Dispersie statică: Adresare deschisă

Fișierul conține doare date (fără pointeri)

Inserarea unei înregistrări cu valoarea cheii K :

Dacă slotul de la adresa $h(K)$ e gol: se memorează înregistrarea

Dacă slotul de la adresa $h(K)$ e ocupat se caută următorul slot liber la adresele $h(K)+1$, $h(K)+2$, ..., $M-1$, 0, ..., $h(K)-1$.

Potrivit pentru ocupare de 75%

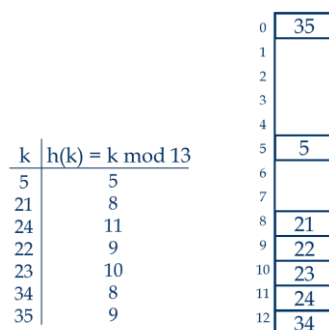


Figura 10.5.

Ștergerea unei înregistrări cu valoarea cheii K :

A. Înlocuirea valorii K cu un cod sau caracter special (vor trebui ajustați algoritmi de căutare și inserare pentru a interpreta corect valorile speciale)



Ștergerea unei înregistrări cu valoarea cheii K:

A. Înlocuirea valorii K cu un cod sau caracter special (vor trebui ajustați algoritmi de căutare și inserare pentru a interpreta corect valorile speciale)

B. Se elimină înregistrarea și se fac transferuri de înregistrări astfel (i, j, p sunt adrese de memorie):

i e adresa înregistrării șterse,

nu sunt sloturi goale între i și j

înregistrarea stocată la j trebuie să fie stocată la p

Dacă $i < j$ și:

$i < p \leq j$: -

$j \leq p \leq m-1$: înreg. din j se transferă la i

$0 \leq p \leq i$: înreg. din j se transferă la i

Dacă $j < i$ și:

$0 < p \leq j$: -

$j < p \leq i$: înreg. din j se transferă la i

$i < p \leq m-1$: -

Concluzii asupra dispersiei statice

Partițiile conțin înregistrări.

Dacă se adaugă multe înregistrări, se formează liste de pagini de memorie suplimentare (crește numărul de pagini transferate în memoria internă).

Similar, dacă numărul înregistrărilor descreește semnificativ, este o risipă de spațiu de stocare (sloturile din partiții rămân nealocate).

Dispersie extensibilă și liniară: tehnici dinamice de abordare a dezavantajelor dispersiei statice.

Dispersie extensibilă

Idee: Dacă partițiile primare devin neîncăpătoare, fișierul se reorganizează prin *dublarea* numărului de partiții

Fișierul va conține un *director de pointeri la partiții*,

Dublarea numărului de partiții se realizează prin *dublarea directorului*,

Se distribuie înregistrările doar din partiția ce a devenit neîncăpătoare

Nu există pagini suplimentare!

Funcția de dispersie va fi ajustată corespunzător

Directorul are dimensiunea 4.

Pentru a determina partiția lui r , se iau în considerare ultimii biți din reprezentarea binară a lui $h(r)$ (numărul de biți luați în considerare e dat de *adâncimea globală*).

Dacă $h(r) = 5$ = binar 101, se află în partiția referită de 01.

Inserarea: Dacă partiția e plină, acesta se împarte în două (se alocă o nouă pagină și se redistribuie înregistrări).

Dacă e necesar, directorul se dublează. (doar când *adâncimea globală* devine mai mică decât *adâncimea locală* a noii partiții)

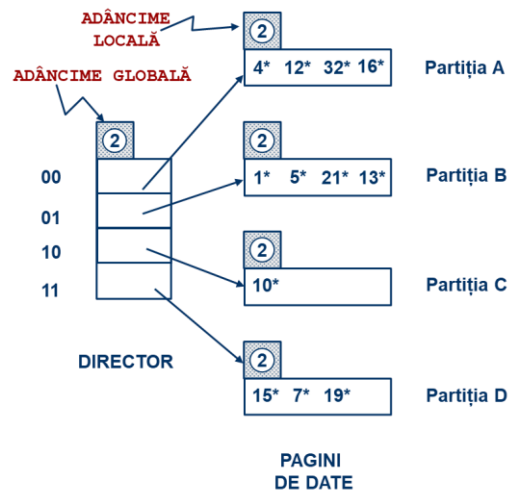
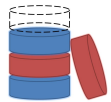


Figura 10.6. Exemplu

Inserare k: $h(k) = 20 \rightarrow$ Dublare

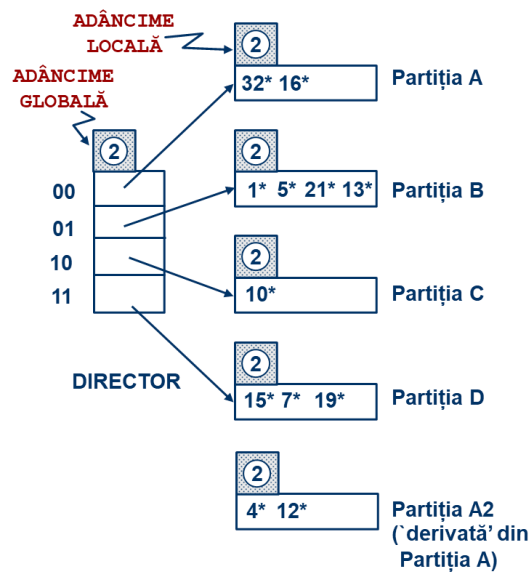


Figura 10.7. Exemplu

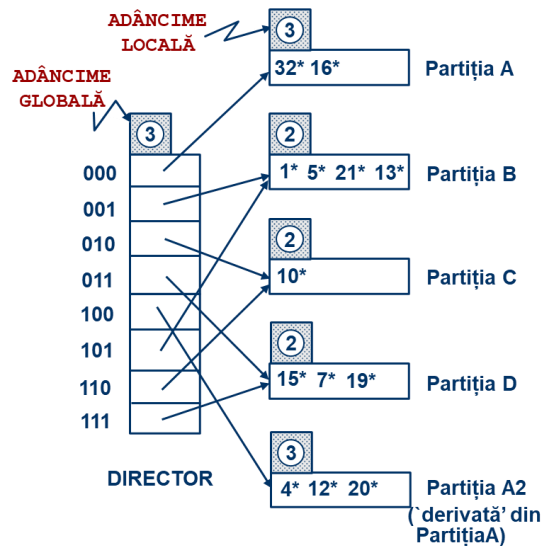


Figura 10.8. Exemplu

Inserare $h(k) = 20$ (binar 10100). Ultimii 2 biți (00) arată că r aparține lui A sau A2. E nevoie de ultimii 3 biți pentru a identifica partiția potrivită.

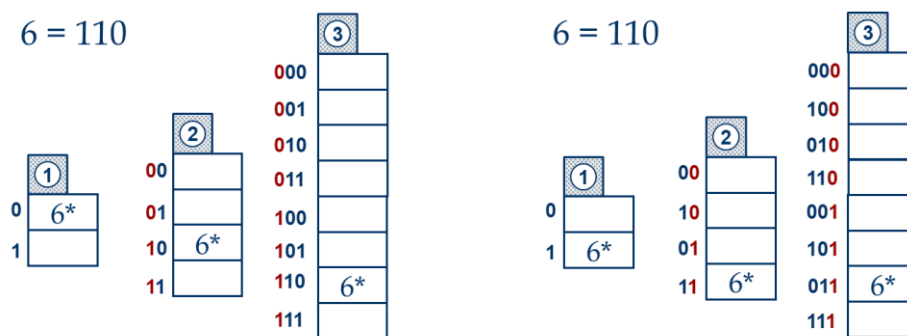
Adâncimea globală a directorului: numărul maxim de biți necesari pentru a determina partiția căreia îi aparține o înregistrare.

Adâncimea locală a partiției: numărul de biți utilizați pentru a determina dacă o înregistrare aparține partiției.

În ce condiții adăugarea unei noi partiții implică dublarea directorului?

Înainte de inserare, *adâncimea locală* partiției = *adâncimea globală*. Inserarea face ca *adâncimea locală* să devină $>$ *adâncimea globală*; directorul se dublează prin copiere!

Utilizarea celor mai puțin semnificativi biți $\rightarrow$ dublarea directorului se realizează prin copiere!



Cel mai puțin semnificativ vs. Cel mai semnificativ

Figura 10.9. Exemplu

Comentarii despre dispersia extensibilă

Dacă directorul încapă în memoria internă, interogările bazate pe egalități au nevoie de un singur acces la disc (suport extern);

Directorul crește brusc iar dacă distribuția valorilor funcției de dispersie nu e una normală, directorul se poate mări excesiv.

Sinonimele multiple pot crea probleme!



Stergeri: Dacă la eliminarea unei înregistrări o partiție se golește, aceasta poate fuziona cu perechea sa (ținând cont de valorile ultimilor biți ai adâncimii locale). Dacă toate adâncimile locale sunt mai mici decât adâncimea globală, directorul se poate înjumătăți.

Evaluarea fișierelor cu organizare directă

Avantaje:

Acces foarte rapid

Cea mai bună soluție pentru *interogări cu egalități*

SELECT * FROM R WHERE A = k

deoarece (în general) indiferent de numărul de înregistrări adăugate sau șterse, răspunsul la interogare va fi dat de citirea unei singure pagini de memorie de pe suportul extern.

Dezavantaje:

Ordinea secvențială a înregistrărilor nu are nici o semnificație

Fișierele cu organizare directă sunt deseori fragmentate.

Nu suportă interogări cu intervale.

Nu sunt recomandate atunci când câmpul transmis ca parametru funcției de dispersie este actualizat frecvent

11. Structura fizică a bazelor de date

SGBD-urile stochează informația pe disc magnetic

Acest lucru are implicații majore în proiectarea unui SGBD!

READ: transfer date de pe disc în memoria internă

WRITE: transfer date din memoria internă pe disc

Ambele operații sunt costisitoare, comparativ cu operațiile *in-memory*, deci trebuie planificate corespunzător!

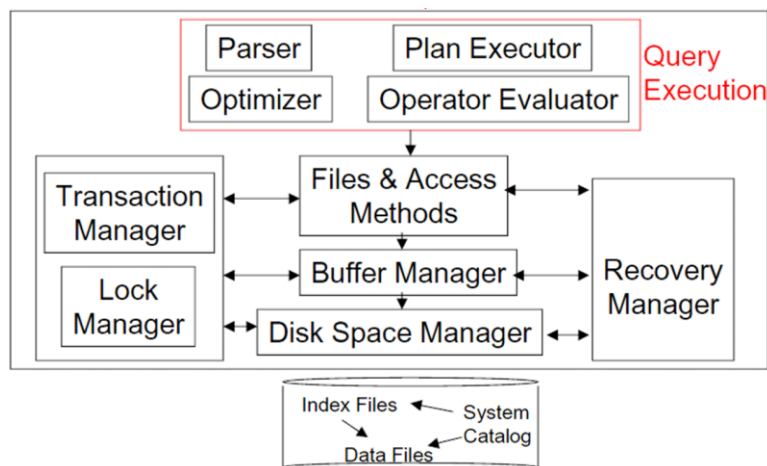


Figura 11.1. Structura internă a unui Sistem de Gestiune a Bazelor de Date



De ce nu stocăm totul în memoria internă?

Răspuns (tipic):

Costă prea mult

Memoria internă este volatilă (datele trebuie să fie persistente)

Procedură tipică (“ierarhie de stocare”)

RAM – pentru datele utiliz. curent (*primary storage*)

Hard-disks – pentru baza de date (*secondary storage*)

Bandă – pentru arhivarea versiunilor anterioare ale datelor (*tertiary storage*)

Legea lui Moore. Gordon Moore: “*Integrated circuits are improving in many ways, following an exponential curve that doubles every 18 months*”

Viteza procesoarelor

Numărul de biți de pe un chip

Numărul de octeți (*bytes*) pe un hard disk

Parametrii ce NU urmează legea lui Moore:

Viteza de accesare a datelor în memoria internă

Viteza de rotație a discului

⇒ Latența devine progresiv mai mare

Timpul de transfer între nivelele ierarhiei mediilor de stocare este tot mai mare în comparație cu timpul de calcul

Discuri magnetice

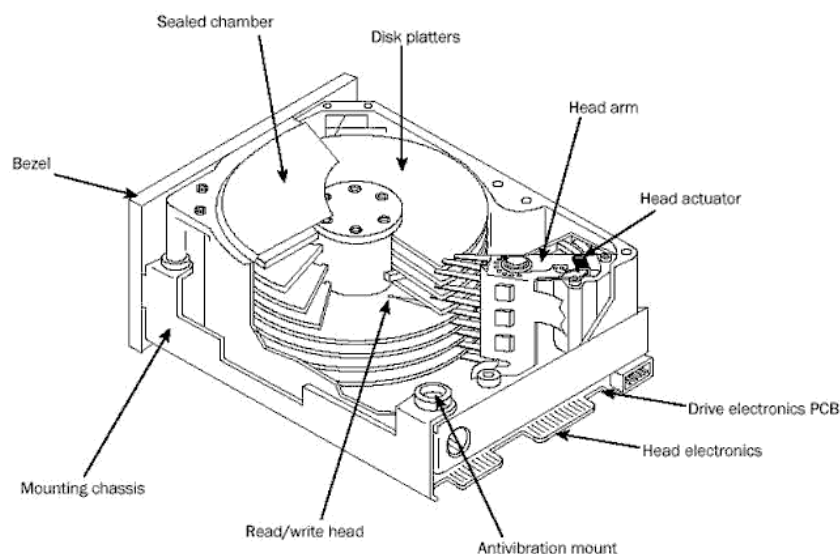
Utilizat ca mediu de stocare secundar

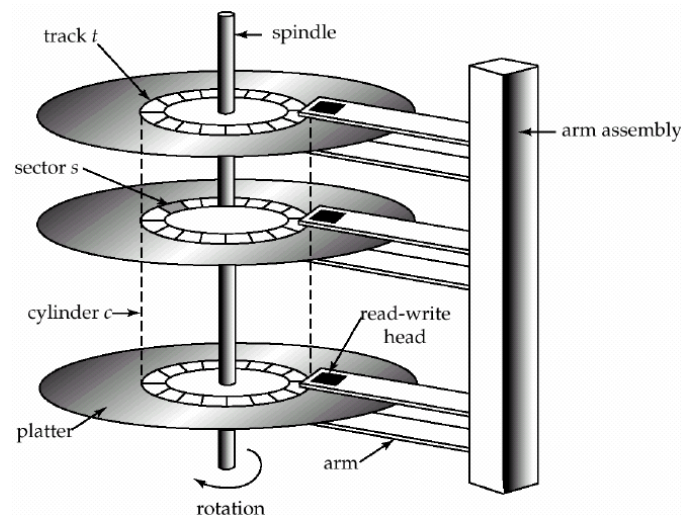
Avantaj major asupra benzilor: *acces direct*

Datele sunt stocate și citite în unități numite blocuri sau pagini.

Spre deosebire de memoria internă, timpul de transfer blocurilor/paginilor variază în funcție de poziția acestora pe disc.

! Poziția relativă a paginilor pe disc are un impact major asupra performanței unui SGBD!





Componentele unui disc dur (*hard disk*)

Rotația platanelor (90rps)

Ansamblu de brațe ce se deplasează pentru poziționarea capului magnetic pe pista dorită.

Pistele aflate la aceeași distanță de centrul platanelor formează un cilindru (imaginar!).

Un singur cap citește/ scrie la un moment dat.

Un bloc e un multiplu de sectoare (care e fix).

Accesarea unei pagini (bloc)

Timp de acces (citire/scriere) a unui bloc:

seek time (mutare braț pentru poziționarea capului de citire/scriere pe pistă)

rotational delay (timp poziționare bloc sub cap)

transfer time (transfer date de pe/pe disc)

Seek time și *rotational delay* domină.

Seek time variază între 1 și 20msec

Rotational delay variază între 0 și 10msec

Transfer rate e de aproximativ 1msec pe 4KB (pagină)

Reducerea costului I/O: reducere *seek/rotational delays*!

Soluții *hardware* sau *software*?

Aranjarea paginilor/blocurilor pe disc

Conceptul de *next block*:

blocuri pe aceeași pistă, urmate de

blocuri pe același cilindru, urmate de

blocuri pe cilindri adiacenți

Blocurile dintr-un fișier trebuie dispuse secvențial pe disc (*next*), pentru a minimiza *seek delay* și *rotational delay*.

În cazul unei scanări secvențiale, citirea de pagini în avans (*pre-fetching*) este esențială!

RAID

Disk Array: configurație de discuri magnetice ce abstractizează un singur disc.

mult mai puțin costisitor; se utilizează mai multe discuri de capacitate mică și ieftine în locul unui disc de capacitate ridicată

Scop: Creșterea performanței și fiabilității.

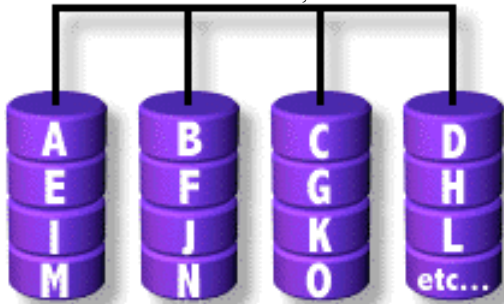
Tehnici:



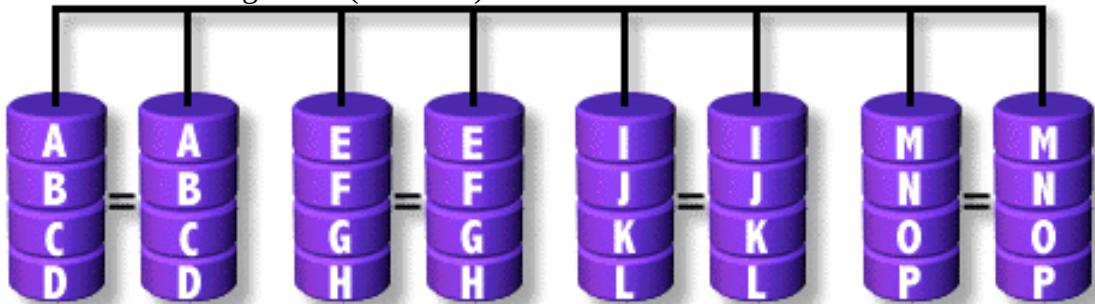
Data striping: distribuirea datelor pe mai multe discuri (în partiții prestabilite - *striping unit*)

Mirroring: stocarea automată a unei copii a datelor pe alte discuri → redundanță. Permite reconstruirea datelor în cazul unor defecte ale discurilor.

Nivel 0: Fără redundanță



Nivel 1: Discuri oglindite (*mirrored*)



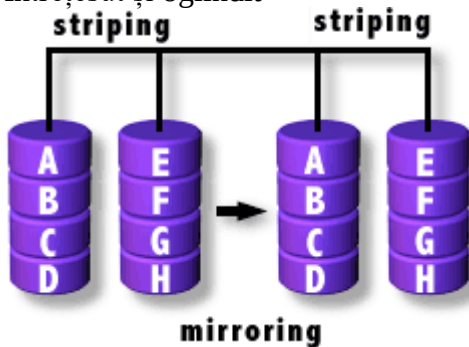
Fiecare disc are o “oglindă” (*check disk*)

Citiri paralele, o scriere implică două discuri.

Rata maximă de transfer = rata de transfer a unui disc

Nivel 0+1:

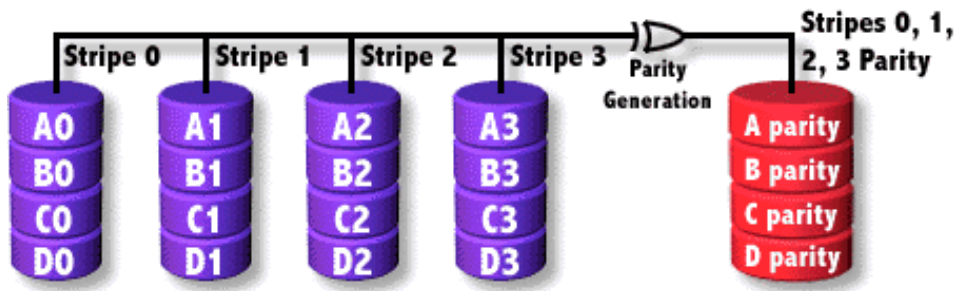
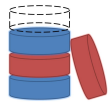
Întrețesut și oglindit



Nivel 3: Bit de paritate intercalat

Striping Unit: un bit. un singur disc de verificare

Fiecare citire și scriere implică toate discurile



Nivel 4: Bloc de paritate

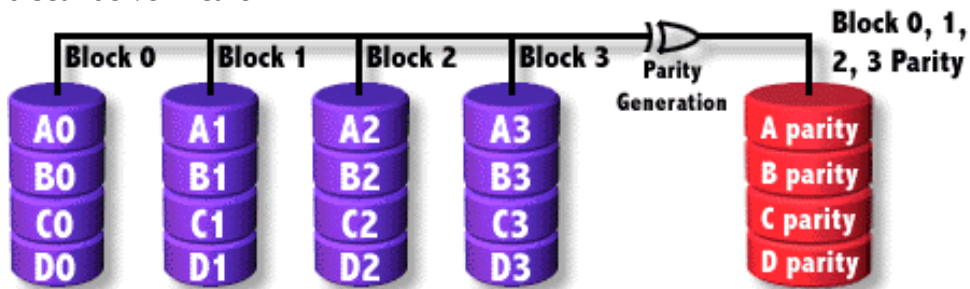
Striping unit: un bloc, un singur disc de verificare.

Citiri în paralel pentru cereri de dimensiune mică

Scrierile implică

blocul modificat și

discul de verificare



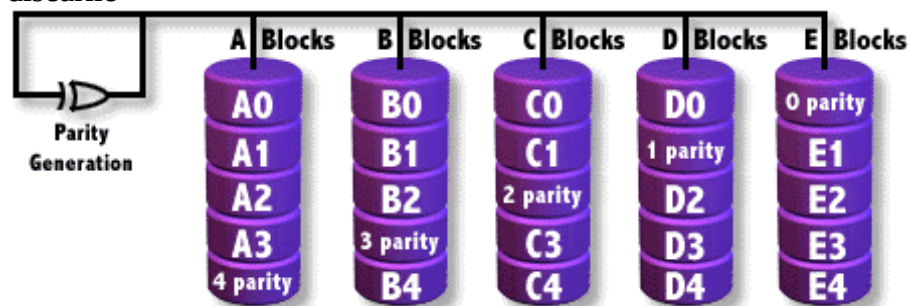
Nivel 5: Bloc de paritate distribuit

Similar cu RAID 4,

dar blocurile de paritate

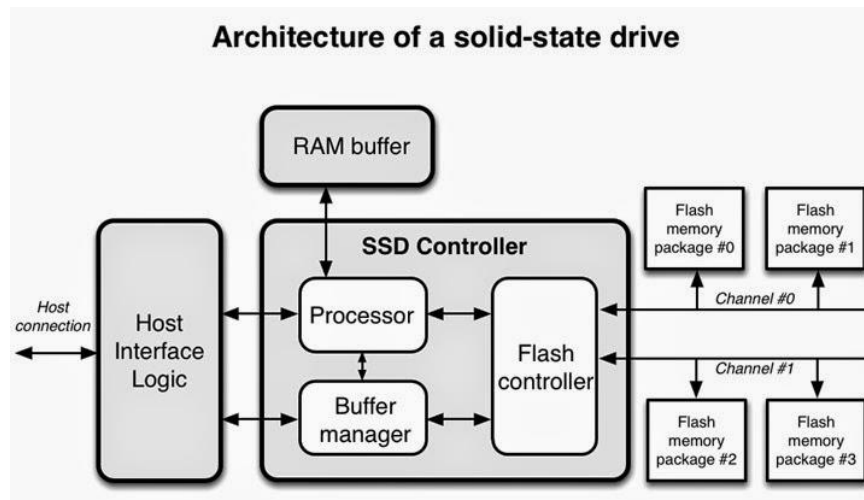
sunt distribuite pe toate

discurile



Solid State Drive

SSD-urile conțin mai multe componente NAND flash (16/32 GB)



NAND Flash Media e format din mai multe celule NAND aranjate pe planuri multiple: aceste planuri permit accesul paralel la NAND
permit, de asemenea, întreținerea datelor
Datele sunt transferate printr-un *cache*

SSD – Avantaje

Latență foarte mică

seek time este zero

Viteze mari de citire și scriere

Mai robuste fizic

Rezistente la șocuri

Zero părți mobile

Silențioase

Consumă puțin

Excelează la citiri/scrieri de dimensiuni reduse

“Imun” la fragmentarea datelor

SSD – Dezavantaje

Cost per GB mult mai mare comparativ cu discurile magnetice

Dimensiuni

HDD 3.5” cu 4TB sunt relativ comune

SSD 3.5” cu 2TB sunt disponibile (rare și scumpe)

Cicluri de citire/scriere limitate

1 - 2 milioane cicluri de scriere ⇒ uzura MLC

(multi-level cell)

Sub 5 milioane cicluri de scriere ⇒ uzura SLC

Gestionare *buffer* (zona de lucru)

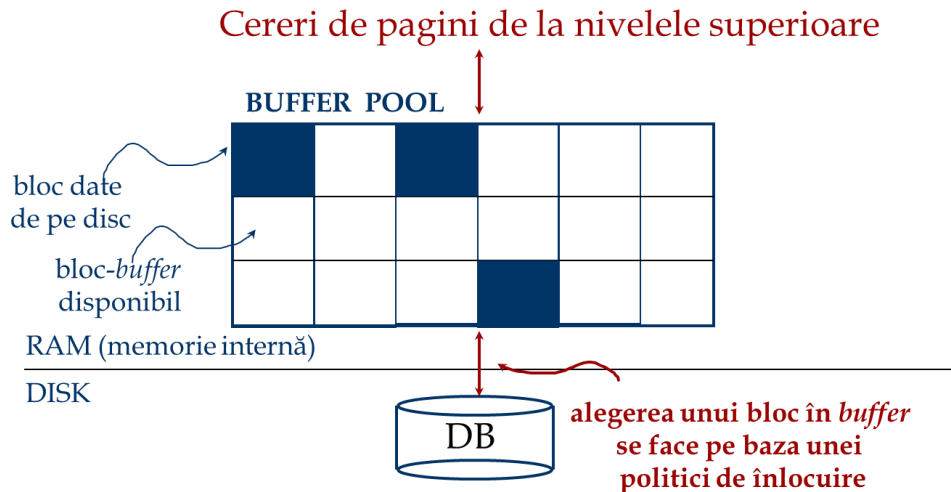
de către SGBD

Buffer – porțiune a memoriei interne utilizată pentru stocarea de copii ale blocurilor de date.

Buffer manager – modul SGBD responsabil cu alocarea spațiului de *buffer* în memoria internă.

Buffer manager-ul este apelat când este necesară accesarea unui bloc de pe disc

SGBD-ul operează asupra datelor din memoria internă

Gestionare *buffer* de către SGBD

Se actualizează o tabelă cu perechi
<nr_bloc_buffer, id_bloc_date>

La cererea unui bloc de date...

Dacă blocul nu se regăsește în *buffer*:

Se alege un bloc disponibil pt. înlocuire

Dacă blocul conține modificări acesta este transferat pe disc

Se citește blocul dorit în locul vechiului bloc

Blocul e *fixat* și se returnează adresa sa

! Dacă cererile sunt predictibile (ex. scanări secvențiale) pot fi citite în avans mai multe blocuri la un moment dat

Gestionare *buffer* de către SGBD

Programul care a cerut blocul de date trebuie să îl elibereze și să indice dacă blocul a fost modificat:

folosește un dirty bit.

Același bloc de date din *buffer* poate fi folosit de mai multe programe:

folosește un pin count. Un bloc-*buffer* e un candidat pentru a fi înlocuit dacă pin count = 0.

Modulele *Concurrency Control* & *Crash Recovery* pot implica acțiuni I/O adiționale la înlocuirea unui bloc-*buffer*.

Politici de înlocuire a blocurilor în *buffer*

Least Recently Used (LRU): utilizează șablonul de utilizare a blocurilor ca predictor al utilizării viitoare. Interogările au șabloane de acces bine definite (ex, scanările secvențiale), iar un SGBD poate utiliza informațiile din interogare pentru a prezice accesările ulterioare ale blocurilor.

Toss-immediate: eliberează spațiul ocupat de un bloc atunci când a fost procesat ultima înregistrare stocată în blocul respectiv

Most recently used (MRU): după procesarea ultimei înregistrări dintr-un bloc, blocul este eliberat (*pin count* e decrementat) și devine blocul utilizat cel mai recent.

Politici de înlocuire a blocurilor în *buffer*



Buffer Manager poate utiliza **informații statistice** cu privire la probabilitatea ca o anumită cerere să refere un anumit bloc sau chiar o anumită relație

Politicile de înlocuire pot avea un impact determinant în ceea ce privește numărul de I/Os – dependent de șablonul de acces.

Sequential flooding: problemă generată de LRU + scanări secvențiale repetate.

Nr blocuri-buffer < Nr blocuri în tabelă → fiecare cerere de pagină determină un I/O. MRU e preferabil într-o astfel de situație.

SGBD vs. Sistemul de fișiere al SO

SO gestionează spațiul pe disc și *buffer*-ul.

De ce o face și un SGBD?

Diferențe de suport oferit de SO: probleme de portabilitate

Existența unor limitări (ex. fișierele nu pot fi salvate pe mai multe discuri)

Gestionarea *buffer*-ului de SGBD presupune abilitatea de a:

fixa/elibera blocuri, forța salvarea unui bloc pe disc (important pentru *concurrency control* & *crash recovery*),

ajustarea *politicii de înlocuire* și citirea de blocuri în avans pe baza șablonului de acces ale operațiilor tipice BD.

Fișiere de înregistrări

Modulele de nivel superior ale unui SGBD operează cu înregistrări și fișiere de înregistrări, nu cu pagini sau blocuri

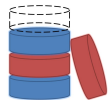
Fișier = colecție de pagini; fiecare pagină conține o colecție de înregistrări; trebuie să permită:

inserarea/ștergerea/modificare înregistrărilor

citirea unei înregistrări particulare (folosind un record id)

scanarea tuturor înregistrărilor (eventual filtrate)

O pagină ce conține o înregistrare poate fi identificată prin intermediul referinței acestuia (rid)



Bibliografie

1. Aho, A., Hopcroft, J., Ullman, J., Data Structures and Algorithms. Addison-Wesley, Reading, Massachusetts, 1983.
2. Date, C.J., An Introduction to Data Base Systems. Addison Wesley, Reading, MA, 1995.
3. Davidson, L., Moss, J. - Pro SQL Server 2012 Relational Database Design and Implementation, Apress, 2012
4. Helman, P., The Science of Database Management. Richard D. IRWIN, Inc., 1994.
5. Korth, H.F., Silberschatz, A., Data Base System Concepts. McGraw-Hill Book Compagny, 1986.
6. Knuth, D.E., Tratat de programare a calculatoarelor. Algoritmi fundamentali. Ed.Tehnica, Bucuresti 1974.
7. Knuth, D.E., Tratat de programare a calculatoarelor. Sortare si cautare. Ed.Tehnica, Bucuresti 1976.
8. Pratt, Philip J., Last, Mary Z. - A Guide to SQL, Cengage Learning, 2014
9. Ramakrishnan, R., Database Management Systems. McGraw-Hill, 1998.